



Chapter 8

Matrix Factorization

“He who knows only his own side of the case knows little of that. His reasons may be good, and no one may have been able to refute them. But if he is equally unable to refute the reasons on the opposite side, if he does not so much as know what they are, he has no ground for preferring either opinion.”—John Stuart Mill

8.1 Introduction

Just as multiplication can be generalized from scalars to matrices, the notion of factorization can also be generalized from scalars to matrices. Exact matrix factorizations need to satisfy the size and rank constraints that are imposed on matrix multiplication. For example, when an $n \times d$ matrix A is factorized into two matrices B and C (i.e., $A = BC$), the matrices B and C must be of sizes $n \times k$ and $k \times d$ for some constant k . For exact factorization to occur, the value of k must be equal to at least the rank of A . This is because the rank of A is at most equal to the minimum of the ranks of B and C . In practice, it is common to perform *approximate* factorization with much smaller values of k than the rank of A .

As in scalars, the factorization of a matrix is not unique. For example, the scalar 12 can be factorized into 2 and 6, or it can be factorized into 3 and 4. If we allow real factors, there are an infinite number of possible factorizations of a given scalar. The same is true of matrices, where even the sizes of the factors might vary. For example, consider the following factorizations of the same matrix:

$$\begin{bmatrix} 3 & 6 \\ 3 & 6 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 3 & 6 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$$

It is clear that a given matrix can be factorized in an unlimited number of ways. However, factorizations with certain types of properties are more useful than others. There are two types of properties that are commonly desired in decompositions:

1. *Linear algebra properties with exact decomposition:* In these cases, one tries to create decompositions in which the individual components of the factorization have specific linear algebra/geometric properties such as orthogonality, triangular nature of the

matrix, and so on. These types of properties are useful for various linear algebra applications like basis construction. All the decompositions that we have seen so far, such as LU decomposition, QR decomposition, and SVD, have linear algebra properties.

2. *Optimization and compression properties with approximate decomposition:* In these cases, one is attempting to factorize a much larger matrix into two or more smaller matrices. Truncated SVD is an example of this type of factorization. Consider the $n \times d$ matrix D , which is truncated to rank- k to create the following factorization:

$$D \approx Q_k \Sigma_k P_k^T \quad (8.1)$$

Here, Q_k is an $n \times k$ orthogonal matrix, Σ_k is a $k \times k$ diagonal matrix with nonnegative entries, and P_k is a $d \times k$ orthogonal matrix. The total number of entries in all three matrices is $(n + d + k)k$, which is often much smaller than the nd entries in the original matrix for large values of n and d . For example, if $n = d = 10^6$ and $k = 1000$, the number of entries in D is 10^{12} , whereas the total number of entries in the factorized matrices is approximately 2×10^9 , which is only 0.2% of the original number of entries.

Singular value decomposition is one of the few factorizations that is useful *both* in terms of its linear algebra properties (when used in exact form), and in terms of its compression properties (when used in truncated form). The value k is referred to as the *rank* of the factorization. The optimization view of matrix factorization $D \approx UV^T$ is particularly useful in machine learning by instantiating D , U , and V as follows:

1. When D is a *document-term matrix* containing frequencies of words (columns of D) in documents (rows of D), the rows of U provide latent representations of documents, whereas the rows of V provide latent representations of words.
2. A rating is a numerical score that a user gives to an item (e.g., movie). Recommender systems collect ratings of users for items in order to make predictions of ratings for items they have not yet evaluated. When D is a user-item matrix of ratings, the rows correspond to users and the columns correspond to items. The entries of D contain ratings. Matrix factorization decomposes the *incomplete* matrix $D \approx UV^T$ using only observed ratings. The rows of U provide latent representations of users, whereas the rows of V are the latent representations of items. The matrix UV^T reconstructs the entire ratings matrix (including predictions for missing ratings).
3. Let $D \approx UV^T$ be a graph adjacency matrix, so that the (i, j) th entry of D contains the weight of edge between nodes i and j . In such a case, the rows of both U and V are the latent representations of nodes. The latent representations of U and V can be used for applications like clustering and *link prediction* (cf. Chapters 9 and 10).

In the optimization-centric view, one can impose specific properties on the decomposed matrices as constraints of the optimization problem (such as nonnegativity of matrix entries). These specific properties are often useful in various types of applications.

This chapter is organized as follows. The next section provides an overview of the optimization-centric view to matrix factorization. Unconstrained matrix factorization methods are discussed in Section 8.3. Nonnegative matrix factorization methods are introduced in Section 8.4. Weighted matrix factorization methods are introduced in Section 8.5. Logistic and maximum-margin matrix factorizations are discussed in Section 8.6. Generalized low-rank models are introduced in Section 8.7. Methods for shared matrix factorization are discussed in Section 8.8. Factorization machines are discussed in Section 8.9. A summary is given in Section 8.10.

8.2 Optimization-Based Matrix Factorization

The optimization-centric view of matrix factorization is at the core of its usefulness in machine learning applications. The optimization-centric view creates a *compressed* representation of the matrix, which is always helpful in getting rid of the random artifacts and generalizing predictions of missing values from seen data to unseen data. After all, repeated patterns in the data, which are useful for predictions of missing values in new data instances, are retained in a compressed representation.

In the following, we discuss a two-way factorization of an $n \times d$ matrix D into an $n \times k$ matrix U and a $d \times k$ matrix V , although any two-way factorization can be converted into a three-way factorization $Q\Sigma P^T$ (like SVD) using the approach in Section 7.2.7 of Chapter 7. The main goal of the factorization is to create an objective function so that UV^T can be used to reconstruct the original matrix D . Most forms of optimization-centric matrix factorization are special cases of the following optimization model over matrices U and V :

$$\begin{aligned} &\text{Maximize similarity between entries of } D \text{ and } UV^T \\ &\text{subject to:} \\ &\text{Constraints on } U \text{ and } V \end{aligned}$$

Constraints are used to ensure specific properties of the factor matrices. A commonly used constraint is that of nonnegativity of the matrices U and V . The simplest possible objective function, which is used in SVD, is $\|D - UV^T\|_F^2$. Other objective functions such as *log-likelihood* and *I-divergence* are also used to create probabilistic models. Most matrix factorization formulations are not convex; nevertheless, gradient descent works quite well in these cases.

In some cases, it is possible to weight specific matrix entries in the objective function. In fact, for certain types of matrices, it makes more sense to interpret the entry of the matrix as a weight. This is common in the case of *implicit feedback data* in recommender systems, where all entries are assumed to be binary, and the values of non-zero entries are treated as weights. For example, a matrix containing the quantities of sales of products (column identifiers) to various users (row identifiers) is always assumed to be binary depending on whether or not users have bought the products. This approach is also sometimes used with frequency matrices in the text domain [101].

Logistic matrix factorization methods apply a logistic function on the entries of UV^T in order to materialize the probability that a particular entry is 1. Such an approach works well for matrices in which the nonnegative values should be treated as frequencies of binary values. The basic idea here is to assume that the entries of D are frequencies obtained by repeatedly sampling each entry in the matrix with probabilities present in the matrix $P = \text{Sigmoid}(UV^T)$. The sigmoid function is defined as follows:

$$\text{Sigmoid}(x) = \frac{1}{1 + \exp(-x)}$$

Note that D and P are two matrices of the same size, and the frequencies in D will be roughly proportional to the entries in P :

$$D \sim \text{Instantiation of frequencies obtained by sampling from } P$$

The optimization model maximizes a log-likelihood function based on this probabilistic model. A surprisingly large number of applications in machine learning can be shown to be special cases of matrix factorization, especially if one is willing to incorporate complex

objective functions and constraints in the factorization. Matrix factorization is used for feature engineering, clustering, kernel methods, link prediction, and recommendations. In each case, the secret is to choose an appropriate objective function and corresponding constraints for the problem at hand. As a specific example, we will show that the k -means algorithm is a special case of matrix factorization, albeit with some special constraints.

8.2.1 Example: K-Means as Constrained Matrix Factorization

The k -means algorithm determines a set of k centroids, so that the sum-of-squared errors of each row in an $n \times d$ data matrix D from the closest centroid is minimized. This algorithm can be shown to be a special case of matrix factorization if one is willing to incorporate additional constraints in the factorization. Consider the following optimization problem with $n \times k$ and $d \times k$ factor matrices U and V , respectively:

$$\begin{aligned} &\text{Minimize}_{U,V} \|D - UV^T\|_F^2 \\ &\text{subject to:} \\ &\text{Columns of } U \text{ are mutually orthogonal} \\ &u_{ij} \in \{0, 1\} \end{aligned}$$

This is a *mixed integer matrix factorization problem*, because the entries of U are constrained to be binary values. An equivalent optimization formulation is given in Section 4.10.3 of Chapter 4. In this case, each row of U can be shown to contain exactly a single 1, corresponding to the cluster membership of that row. Each column of V contains the d -dimensional centroid of one of the k clusters. As discussed in Section 4.10.3 of Chapter 4, this optimization problem can be solved using block coordinate descent, *which is identical to the k -means algorithm*. The fact that k -means is a special case of matrix factorization is an example of the fact that the family of matrix factorization methods is extremely expressive in its relationship to a wide variety of machine learning methods. This chapter will, therefore, explore multiple ways of performing matrix factorization together with their applications.

8.3 Unconstrained Matrix Factorization

The problem of unconstrained matrix factorization is defined as follows:

$$\text{Minimize}_{U,V} J = \frac{1}{2} \|D - UV^T\|_F^2$$

Here, D , U , and V are matrices of sizes n , d , and k , respectively. The value of k is typically much smaller than the rank of the matrix D . This problem, which is discussed in Section 7.3.2 of Chapter 7, can be shown to provide the same solution as SVD. As discussed in Chapter 7, the top eigenvectors of $D^T D$ provide the columns of V , and the top eigenvectors of DD^T provide the columns of U . The columns of V are unit normalized, whereas those of U are normalized so that the norm of the i th column is equal to the i th singular value of D .

However, many alternative optima are possible. For example, even the normalization of the columns is not unique. Instead of normalizing columns of V to unit norm, one could easily normalize the columns of U to unit norm, and adjust the normalization of each column of V appropriately. The main point is that the product of the norms of the i th columns of U and V needs to be the i th singular value of D . Furthermore, the columns of U and V need

not be orthonormal sets for an optimum solution to exist. Given an optimum pair (U_0, V_0) , one could change the basis of the column space of V_0 to a non-orthogonal one and adjust U_0 to the corresponding coordinates in the non-orthogonal basis system, so that the product $U_0 V_0^T$ does not change. To understand this point, we recommend the reader to solve the following problem:

Problem 8.3.1 *Let $D \approx Q_k \Sigma_k P_k^T$ be the rank- k SVD of D . The results in Chapter 7 show that $(U, V) = (Q_k \Sigma_k, P_k)$ represents an optimal pair of rank- k solution matrices to the unconstrained matrix factorization problem that is posed in this section. Show that $(U, V) = (Q_k \Sigma_k R_k^T, P_k R_k^{-1})$ is an alternative optimal solution to the unconstrained matrix factorization problem for any $k \times k$ invertible matrix R_k .*

8.3.1 Gradient Descent with Fully Specified Matrices

In this section, we will investigate a method that finds a solution to the unconstrained optimization problem with the use of gradient descent. This approach does not guarantee the orthogonal solutions provided by singular value decomposition; however, the formulation is equivalent and should (ideally) lead to a solution with the same objective function value. The approach also has the advantage that it can easily be adapted to more difficult settings such as the presence of missing values in the matrix. A natural application of this type of approach is that of matrix factorization in recommender systems. Recommender systems use the same optimization formulation as SVD; however, the resulting basis vectors of the factorization are not guaranteed to be orthogonal.

In order to perform gradient descent, we need to compute the derivative of the unconstrained optimization problem with respect to the parameters in the matrices $U = [u_{iq}]$ and $V = [v_{jq}]$. The simplest approach is to compute the derivative of the objective function J with respect to each parameter in the matrices U and V . First, the objective function is expressed in terms of the individual entries in the various matrices. Let the (i, j) th entry of the $n \times d$ matrix D be denoted by x_{ij} . Then, the objective function can be restated in terms of the entries of the matrices D , U , and V as follows:

$$\text{Minimize } J = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^d \left(x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right)^2$$

The quantity $e_{ij} = x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js}$ is the error of the factorization for the (i, j) th entry. Note that the objective function J minimizes the sum of squares of e_{ij} . One can compute the partial derivative of the objective function with respect to the parameters in the matrices U and V as follows:

$$\begin{aligned} \frac{\partial J}{\partial u_{iq}} &= \sum_{j=1}^d \left(x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right) (-v_{jq}) \quad \forall i \in \{1 \dots n\}, q \in \{1 \dots k\} \\ &= \sum_{j=1}^d (e_{ij}) (-v_{jq}) \quad \forall i \in \{1 \dots n\}, q \in \{1 \dots k\} \\ \frac{\partial J}{\partial v_{jq}} &= \sum_{i=1}^n \left(x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right) (-u_{iq}) \quad \forall j \in \{1 \dots d\}, q \in \{1 \dots k\} \\ &= \sum_{i=1}^n (e_{ij}) (-u_{iq}) \quad \forall j \in \{1 \dots d\}, q \in \{1 \dots k\} \end{aligned}$$

One can also express these derivatives in terms of matrices. Let $E = [e_{ij}]$ be the $n \times d$ matrix of errors. In the denominator layout of matrix calculus, the derivatives can be expressed as follows:

$$\begin{aligned}\frac{\partial J}{\partial U} &= -(D - UV^T)V = -EV \\ \frac{\partial J}{\partial V} &= -(D - UV^T)^T U = -E^T U\end{aligned}$$

The above matrix calculus identity can be verified by using the relatively tedious process of expanding the (i, q) th and (j, q) th entries of each of the above matrices on the right-hand side, and showing that they are equivalent to the (corresponding) scalar derivatives $\frac{\partial J}{\partial u_{iq}}$ and $\frac{\partial J}{\partial v_{jq}}$. An alternative approach that directly uses the matrix calculus identities of Chapter 4 is given in Figure 8.1. The reader may choose to skip over this derivation without loss of continuity.

The optimality conditions for this optimization problem are therefore obtained by setting these derivatives to 0. Therefore, we obtain the optimality conditions $DV = UV^T V$ and $D^T U = VU^T U$. These optimality conditions can be shown to hold for the solution obtained from SVD $U = Q_k \Sigma_k$ and $V = P_k$.

Problem 8.3.2 Let $Q_k \Sigma_k P_k^T$ be the rank- k truncated SVD of matrix D . Show that the solution $U = Q_k \Sigma_k$ and $V = P_k$ satisfies the optimality conditions $DV = UV^T V$ and $D^T U = VU^T U$.

A useful hint for solving the above problem is to use the spectral decomposition of SVD as a sum of rank-1 matrices.

Although the optimality condition leads to the standard SVD solution $D \approx [Q_k \Sigma_k] P_k^T$, one can also find an optimal solution by using gradient descent. The updates for gradient descent are as follows:

$$\begin{aligned}U &\Leftarrow U - \alpha \frac{\partial J}{\partial U} = U + \alpha EV \\ V &\Leftarrow V - \alpha \frac{\partial J}{\partial V} = V + \alpha E^T U\end{aligned}$$

Here, $\alpha > 0$ is the learning rate.

The optimization model is identical to that of SVD. If the aforementioned gradient descent method is used (instead of the power iteration method of the previous chapter), one will typically obtain solutions that are equally good in terms of objective function value, but for which the columns of U (or V) are not mutually orthogonal. The power iteration methods yields solutions with orthogonal columns. Although the standardized SVD solution with orthonormal columns is typically not obtained by gradient descent, the k columns of U will span¹ the same subspace as the columns of Q_k , and the columns of V will span the same subspace as the columns of P_k .

The gradient-descent approach can be implemented efficiently when the matrix D is sparse by sampling entries from the matrix for making updates. This is essentially a *stochastic* gradient descent method. In other words, we sample an entry (i, j) and compute its error

¹This will occur under the assumption that the top- k eigenvalues of $D^T D$ are distinct. Tied eigenvalues result in a non-unique solution for SVD, which might sometimes result in some differences in the subspace corresponding to the smallest eigenvalue within the rank- k solution.

Consider the following objective function the $n \times d$ matrix D with rank- k matrices U and V :

$$J = \frac{1}{2} \|D - UV^T\|_F^2$$

Matrix calculus can be used for computing derivatives with respect to U and V after decomposing the Frobenius norm into row-wise vector norms or column-wise vector norms, depending on whether we wish to compute the derivative of J with respect to U or V . Let $\bar{X}_i$ be the i th row of D (row vector), $\bar{d}_j$ be the j th column of D (column vector), $\bar{u}_i$ be the i th row of U (row vector), and $\bar{v}_j$ be the j th row of V (row vector).

Then, the Frobenius norm can be decomposed in row-wise fashion as follows:

$$J = \frac{1}{2} \sum_{i=1}^n \|\bar{X}_i - \bar{u}_i V^T\|^2 = \frac{1}{2} \underbrace{\sum_{i=1}^n \bar{X}_i \bar{X}_i^T}_{\text{Constant}} - \sum_{i=1}^n \bar{X}_i V \bar{u}_i^T + \frac{1}{2} \sum_{i=1}^n \bar{u}_i V^T V \bar{u}_i^T$$

To compute the derivative of the two non-constant terms with respect to $\bar{u}_i$, we can use identities (i) and (ii) of Table 4.2(a) in Chapter 4. This yields the following:

$$\begin{aligned} \frac{\partial J}{\partial \bar{u}_i^T} &= -V^T \bar{X}_i^T + V^T V \bar{u}_i^T \\ \frac{\partial J}{\partial [\bar{u}_1^T \dots \bar{u}_n^T]} &= -V^T [\bar{X}_1^T \dots \bar{X}_n^T] + V^T V [\bar{u}_1^T \dots \bar{u}_n^T] \\ \frac{\partial J}{\partial U^T} &= -V^T D^T + V^T V U^T \\ \frac{\partial J}{\partial U} &= -DV + UV^T V = -(D - UV^T)V \end{aligned}$$

In order to compute the derivative with respect to V , one will need to decompose the squared Frobenius norm in J in column-wise fashion as follows:

$$J = \frac{1}{2} \sum_{j=1}^d \|\bar{d}_j - U \bar{v}_j^T\|^2 = \frac{1}{2} \underbrace{\sum_{j=1}^d \bar{d}_j^T \bar{d}_j}_{\text{Constant}} - \sum_{j=1}^d \bar{d}_j^T U \bar{v}_j^T + \frac{1}{2} \sum_{j=1}^d \bar{v}_j U^T U \bar{v}_j^T$$

One can again use identities (i) and (ii) of Table 4.2(a) to show the following:

$$\frac{\partial J}{\partial \bar{v}_j^T} = -U^T \bar{d}_j + U^T U \bar{v}_j^T$$

As in the previous case, one can put together the derivatives for different rows of V to obtain the following:

$$\frac{\partial J}{\partial V} = -D^T U + V U^T U = -(D - UV^T)^T U$$

Figure 8.1: Alternative derivation of factorization gradients using matrix calculus

e_{ij} . Subsequently, we make the following updates to the i th row $\bar{u}_i$ of U and the j th row $\bar{v}_j$ of V , which are also referred to as *latent factors*:

$$\begin{aligned}\bar{u}_i &\leftarrow \bar{u}_i + \alpha e_{ij} \bar{v}_j \\ \bar{v}_j &\leftarrow \bar{v}_j + \alpha e_{ij} \bar{u}_i\end{aligned}$$

One cycles through the sampled entries of the matrix (making the above updates) until convergence. The fact that we can sample entries of the matrix for updates means that we do not need fully specified matrices in order to learn the latent factors. This basic idea forms the foundations of recommender systems.

Problem 8.3.3 (Regularized Matrix Factorization) *Let D be an $n \times d$ matrix that we want to factorize using a rank- k decomposition into U and V . Suppose that we add the regularization terms $\frac{\lambda}{2}(\|U\|^2 + \|V\|^2)$ to the objective function $\frac{1}{2}\|D - UV^T\|_F^2$. Show that the gradient descent updates need to be modified as follows:*

$$\begin{aligned}U &\leftarrow U(1 - \alpha\lambda) + \alpha EV \\ V &\leftarrow V(1 - \alpha\lambda) + \alpha E^T U\end{aligned}$$

The entries of the matrices U and V can be initialized as follows. First, all $n \times k$ entries in U are independently sampled from a standard normal distribution, and then each column is normalized to the unit vector. Note that the matrix U contains roughly orthogonal columns, if n is large (see Exercise 18 of Chapter 1). The matrix V is selected to be $D^T U$. This approach ensures that UV^T yields $UU^T D$, where UU^T is (roughly) a projection matrix because of the approximate orthogonality of U . Thus, the initialized product is already closely related to the target matrix.

8.3.2 Application to Recommender Systems

Let D be an $n \times d$ ratings matrix representing the ratings of n users for d items. The (i, j) th entry in the matrix D is denoted by x_{ij} , and it represents the rating of user i for item j . The key distinguishing point of a recommendation application is that the vast majority of ratings are missing. This is because users do not specify the ratings of the vast majority of the items in collaborative filtering applications. An example of a ratings matrix with missing entries is shown in Figure 8.2. In other words, the value of x_{ij} is observed (known) for only a small subset of the entries. The goal of the recommendation problem is to predict the missing ratings from the known ones.

Let S represent the set of indices of all the observed ratings. Therefore, we have:

$$S = \{(i, j) : x_{ij} \text{ is observed}\} \quad (8.2)$$

As in the case of traditional matrix factorization, we would like to factorize the *incomplete* ratings matrix D with the use of only the entries in S . In the terminology of recommender systems, the $n \times k$ matrix U is referred to as the *user factor* matrix, and the $d \times k$ matrix V is referred to as the *item factor matrix*. Regularization is particularly important in the case of the collaborative filtering application because of the paucity of observed data. Therefore, an additional term $\frac{\lambda}{2}(\|U\|_F^2 + \|V\|_F^2)$ is added to the objective function.

Once the user and item factor matrices have been learned, the entire ratings matrix can be reconstructed as UV^T . In practice, we only need to reconstruct the (i, j) th entry of matrix D as follows:

$$\hat{x}_{ij} = \sum_{s=1}^k u_{is} \cdot v_{js} \quad (8.3)$$

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
TOM	1			5		2
JIM		5			4	
JOE	5	3		1		
ANN			3			4
JILL				3	5	
SUE	5		4			

Figure 8.2: A ratings matrix with missing ratings

Note the “hat” symbol (i.e., circumflex) on the rating on the left-hand side to indicate that it is a predicted value rather than an observed value. The error e_{ij} of the prediction is $e_{ij} = x_{ij} - \hat{x}_{ij}$ for ratings that are observed.

One can then formulate the objective function in terms of the *observed* entries in D as follows:

$$\text{Minimize } J = \frac{1}{2} \sum_{(i,j) \in S} \left(x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right)^2 + \frac{\lambda}{2} \sum_{i=1}^n \sum_{s=1}^k u_{is}^2 + \frac{\lambda}{2} \sum_{j=1}^d \sum_{s=1}^k v_{js}^2$$

The main difference from the objective function in the previous section is the use of only observed entries in S for squared error computation, and the use of regularization. As in the previous section, we can compute the partial derivative of the objective function with respect to the various parameters as follows:

$$\begin{aligned} \frac{\partial J}{\partial u_{iq}} &= \sum_{j:(i,j) \in S} (e_{ij})(-v_{jq}) + \lambda u_{iq} \quad \forall i \in \{1 \dots n\}, q \in \{1 \dots k\} \\ \frac{\partial J}{\partial v_{jq}} &= \sum_{i:(i,j) \in S} (e_{ij})(-u_{iq}) + \lambda v_{jq} \quad \forall j \in \{1 \dots d\}, q \in \{1 \dots k\} \end{aligned}$$

One can also define these errors in matrix calculus notation. Let E be an $n \times d$ error matrix, which is defined to be e_{ij} for each observed entry $(i, j) \in S$ and 0 for each missing entry in the ratings matrix. Note that (unlike vanilla SVD), the error matrix E is already sparse because the vast majority of entries are not specified.

$$\begin{aligned} \frac{\partial J}{\partial U} &= -EV + \lambda U \\ \frac{\partial J}{\partial V} &= -E^T U + \lambda V \end{aligned}$$

Note that the form of the derivative is exactly identical to traditional SVD except for the regularization term and the difference in how the error matrix is defined (to account for missing ratings). Then, the gradient-descent updates for the matrices U and V are as follows:

$$\begin{aligned} U &\Leftarrow U - \alpha \frac{\partial J}{\partial U} = U(1 - \alpha\lambda) + \alpha EV \\ V &\Leftarrow V - \alpha \frac{\partial J}{\partial V} = V(1 - \alpha\lambda) + \alpha E^T U \end{aligned}$$

Here, $\alpha > 0$ is the learning rate. The matrix E can be explicitly materialized as a sparse error matrix, and the above updates can be achieved using only sparse matrix multiplications. Although this approach is referred to as singular value decomposition in the literature on recommender systems (because of the relationship of unconstrained matrix factorization with the SVD optimization model), one will typically not obtain orthogonal columns of U and V with this approach.

8.3.2.1 Stochastic Gradient Descent

Stochastic gradient descent is introduced in Section 4.5.2 of Chapter 4. From the perspective of matrix factorization, the idea is to sample observed entries in S in order to perform the updates one at a time. Note that in the above updates, the (i, j) th error entry e_{ij} influences only the i th rows of U and V respectively. Stochastic gradient descent simply pulls out these updates on an entry-wise basis, rather than summing up the updates over all entries. Let $\bar{u}_i$ be a k -dimensional row vector containing the i th row of U and $\bar{v}_j$ be a k -dimensional row vector containing the j th row of V . Then, stochastic gradient descent cycles through each entry $(i, j) \in S$ in random order, and performs the following updates:

$$\begin{aligned} \bar{u}_i &\Leftarrow \bar{u}_i(1 - \alpha\lambda) + e_{ij}\bar{v}_j \\ \bar{v}_j &\Leftarrow \bar{v}_j(1 - \alpha\lambda) + e_{ij}\bar{u}_i \end{aligned}$$

Here, $\alpha > 0$ is the learning rate. Note that exactly $2k$ entries in the matrices U and V are updated for each observed entry in S . Therefore, a single cycle of stochastic gradient descent through all the observed ratings will make exactly $2k|S|$ updates. One starts by initializing the matrices U and V to uniform random values in $(0, M/\sqrt{k})$, where M is the maximum value of a rating. This type of initialization ensures that the initial product UV^T yields values in a similar order of magnitude as the original ratings matrix. One then performs the aforementioned updates to convergence. Stochastic gradient descent tends to converge faster than gradient descent, and is often the method of choice in recommender systems.

8.3.2.2 Coordinate Descent

The method of coordinate descent is introduced in Section 4.10 of Chapter 4. The basic idea in coordinate descent methods is to optimize one parameter at a time. In the case of matrix factorization, this amounts to optimizing a single parameter in U and V .

In the following, we will use the objective function J used in recommender systems (with incomplete matrices), because it is the more general form of the objective function. All notations such as the observed entry set S and the error e_{ij} are the same as those used in the previous section. By setting the partial derivative of the objective function J with respect to u_{iq} to 0, we obtain the following for each $i \in \{1 \dots n\}$ and $q \in \{1 \dots k\}$:

$$\begin{aligned}
\frac{\partial J}{\partial u_{iq}} &= \sum_{j:(i,j) \in S} (e_{ij})(-v_{jq}) + \lambda u_{iq} = 0 \\
u_{iq}(\lambda + \sum_{j:(i,j) \in S} v_{jq}^2) &= \sum_{j:(i,j) \in S} (e_{ij} + u_{iq}v_{jq})v_{jq} \\
u_{iq} &= \frac{\sum_{j:(i,j) \in S} (e_{ij} + u_{iq}v_{jq})v_{jq}}{\lambda + \sum_{j:(i,j) \in S} v_{jq}^2}
\end{aligned}$$

In the second step of the above algebraic manipulation, we added the quantity $\sum_{j:(i,j) \in S} u_{iq}v_{jq}^2$ to both sides in order to create a stable form of the update. The final form of the algebraic equation contains u_{iq} on both sides, and therefore it provides an iterative update. One can also derive a similar iterative update for each v_{jq} . The updates for the various values of u_{iq} and v_{jq} need to be performed sequentially as follows:

$$\begin{aligned}
u_{iq} &\leftarrow \frac{\sum_{j:(i,j) \in S} (e_{ij} + u_{iq}v_{jq})v_{jq}}{\lambda + \sum_{j:(i,j) \in S} v_{jq}^2} \quad \forall i, q \\
v_{jq} &\leftarrow \frac{\sum_{i:(i,j) \in S} (e_{ij} + u_{iq}v_{jq})u_{iq}}{\lambda + \sum_{i:(i,j) \in S} u_{iq}^2} \quad \forall j, q
\end{aligned}$$

One simply starts with random values of the parameters in the matrices U and V , and performs the above updates. One cycles through the $(m+n) \cdot k$ parameters in U and V with these updates until convergence is reached.

8.3.2.3 Block Coordinate Descent: Alternating Least Squares

Alternating least squares is a form of block coordinate descent, which is introduced in Section 4.10.2 of Chapter 4. This approach with an initial pair of matrices U and V , and then *alternates* between updating U and V , while keeping the other matrix fixed. Therefore, the process works as follows:

1. Keeping U fixed, we solve for each of the d rows of V by treating the problem as a least-squares regression problem. Only the observed ratings in S can be used for building the least-squares model in each case. Let $\bar{v}_j$ be the j th row of V . In order to determine the optimal vector $\bar{v}_j$, we wish to minimize $\sum_{i:(i,j) \in S} (x_{ij} - \sum_{s=1}^k u_{is}v_{js})^2$, which is a least-squares regression problem in $v_{j1} \dots v_{jk}$. The terms $u_{i1} \dots u_{ik}$ are treated as constant values, whereas $v_{j1} \dots v_{jk}$ are treated as optimization variables. Therefore, the k -dimensional vector $\bar{v}_j$ for the j th item are determined with least-squares regression. A total of d such least-squares problems need to be executed, and each least-squares problem has k variables. Because the least-squares problem for each item is independent, this step can be parallelized easily.
2. Keeping V fixed, solve for each of the n rows of U by treating the problem as a least-squares regression problem. Only the specified ratings in S can be used for building the least-squares model in each case. Let $\bar{u}_i$ be the i th row of U . In order to determine the optimal vector $\bar{u}_i$, we wish to minimize $\sum_{j:(i,j) \in S} (x_{ij} - \sum_{s=1}^k u_{is}v_{js})^2$, which is a least-squares regression problem in $u_{i1} \dots u_{ik}$. The terms $v_{j1} \dots v_{jk}$ are treated as constant values, whereas $u_{i1} \dots u_{ik}$ are treated as optimization variables. A total of n such least-squares problems need to be executed, and each least-squares problem has k variables. Because the least-squares problem for each user is independent, this step can be parallelized easily.

Least-squares regression is discussed in Section 4.7 of Chapter 4.

8.4 Nonnegative Matrix Factorization

Nonnegative matrix factorization expresses a matrix as the product of nonnegative factor matrices. Since the product of two nonnegative matrices is nonnegative, it makes sense for the original matrix to be nonnegative as well. Nonnegativity is often satisfied by many real-world matrices containing frequency counts, such as the following:

1. The quantities of items bought by a user are a nonnegative values. The rows of the matrix correspond to users, and the columns correspond to items. The (i, j) th entry corresponds to the number of units bought by user i for item j .
2. The frequencies of various words in a document is a nonnegative value. In this case, the rows correspond to the documents, and the columns correspond to the entire set of words in the lexicon. The (i, j) th entry corresponds to frequency of the j th lexicon word in the i th document.
3. In graph applications, a square adjacency matrix might contain nonnegative weights associated with edges. For example, in the adjacency matrix of a publication network between authors, the entry (i, j) represents the number of times that authors i and j have collaborated with one another.

Why is the nonnegativity of factors useful? As we will see later, the nonnegativity of the factor matrices results in a very high level of interpretability of the factorization. Secondly, the nonnegativity of the factor matrices plays a role in regularizing the factorization. While the error of the factorization always increases by adding constraints such as nonnegativity, the predictions obtained from the factorization often improve for out-of-sample data (such as making predictions with missing data). This is an example of how the goals of optimization in machine learning are often different from those of traditional optimization (cf. Section 4.5.3 of Chapter 4).

8.4.1 Optimization Problem with Frobenius Norm

The most common formulation for nonnegative matrix factorization uses the Frobenius norm as the objective function and imposes a nonnegativity constraints on the factor matrices. Let $D = [x_{ij}]$ be an $n \times d$ data matrix in which the entries are nonnegative. Let U and V be the corresponding $n \times k$ and $d \times k$ factor matrices, so that $D \approx UV^T$. Then, the nonnegative matrix factorization problem is formulated as follows:

$$\begin{aligned} \text{Minimize } J &= \frac{1}{2} \|D - UV^T\|_F^2 + \frac{\lambda}{2} \|U\|_F^2 + \frac{\lambda}{2} \|V\|_F^2 \\ \text{subject to:} \\ U &\geq 0, \quad V \geq 0 \end{aligned}$$

It is evident that this problem differs from unconstrained matrix factorization only in terms of the addition of the nonnegativity constraints. These are *box constraints*, which are particularly easy to address in the context of constrained optimization (cf. Section 6.3.2 of Chapter 6).

8.4.1.1 Projected Gradient Descent with Box Constraints

The optimization problem only has box constraints, which makes it particularly easy to solve. The basic idea is to use the same gradient-descent method as used in unconstrained optimization. Subsequently, the optimization variables are reset to their nonnegative values. In other words, any entry of the matrices U and V , which is negative, is set to 0.

As discussed in Sections 8.3.1 and 8.3.2 on unconstrained matrix factorization, the gradients of the objective function J with respect to the factor matrices U and V are as follows:

$$\begin{aligned}\frac{\partial J}{\partial U} &= -(D - UV^T)V + \lambda U \\ \frac{\partial J}{\partial V} &= -(D - UV^T)^T U + \lambda V\end{aligned}$$

Therefore, the gradient-descent updates (without worrying about the nonnegativity constraints) are as follows:

$$\begin{aligned}U &\Leftarrow U - \alpha \frac{\partial J}{\partial U} = U(1 - \alpha\lambda) + \alpha(D - UV^T)V \\ V &\Leftarrow V - \alpha \frac{\partial J}{\partial V} = V(1 - \alpha\lambda) + \alpha(D - UV^T)^T U\end{aligned}$$

The main difference is that we add two steps to the updates to ensure nonnegativity of each matrix entry:

$$U \Leftarrow \max\{U, 0\}, \quad V \Leftarrow \max\{V, 0\}$$

This procedure is based on the ideas discussed in Section 6.3.2 on box constraints. In practice, projected gradient descent is used rarely for nonnegative matrix factorization.

8.4.2 Solution Using Duality

In this section, we will provide a solution to the nonnegativity problem using Lagrangian relaxation (cf. Section 6.4 of Chapter 6). We convert the inequality constraints to the form $-U \leq 0$ and $-V \leq 0$ in order to be consistent with the conventions used in Section 6.4. Note that there is a constraint for each entry in the matrices U and V , and therefore we need as many Lagrange multipliers as the number of matrix entries. For the (i, s) th entry u_{is} in U , we introduce the Lagrange multiplier $\alpha_{is} \geq 0$, whereas for the (j, s) th entry v_{js} in V , we introduce the Lagrange multiplier $\beta_{js} \geq 0$. One can create a vector $(\bar{\alpha}, \bar{\beta})$ of dimensionality $(n + d) \cdot k$ by putting together all the Lagrangian parameters into a vector. Then, the Lagrangian relaxation is stated as follows:

$$L = \frac{1}{2} \|D - UV^T\|_F^2 + \frac{\lambda}{2} \|U\|_F^2 + \frac{\lambda}{2} \|V\|_F^2 - \sum_{i=1}^n \sum_{r=1}^k u_{ir} \alpha_{ir} - \sum_{j=1}^d \sum_{r=1}^k v_{jr} \beta_{jr} \quad (8.4)$$

The minimax problem of Lagrangian optimization is stated as follows:

$$\text{Max}_{\bar{\alpha} \geq 0, \bar{\beta} \geq 0} \text{Min}_{U, V} L \quad (8.5)$$

As discussed in Section 6.4, the first step is to compute the gradient of the Lagrangian relaxation with respect to the (minimization) optimization variables u_{is} and v_{js} . Therefore, we have:

$$\frac{\partial L}{\partial u_{is}} = -(DV)_{is} + (UV^T V)_{is} + \lambda u_{is} - \alpha_{is} \quad \forall i \in \{1, \dots, n\}, s \in \{1, \dots, k\} \quad (8.6)$$

$$\frac{\partial L}{\partial v_{js}} = -(D^T U)_{js} + (V U^T U)_{js} + \lambda v_{js} - \beta_{js} \quad \forall j \in \{1, \dots, d\}, s \in \{1, \dots, k\} \quad (8.7)$$

These partial derivatives are set to zero in order to obtain the following conditions:

$$-(DV)_{is} + (UV^T V)_{is} + \lambda u_{is} - \alpha_{is} = 0 \quad \forall i \in \{1, \dots, n\}, s \in \{1, \dots, k\} \quad (8.8)$$

$$-(D^T U)_{js} + (V U^T U)_{js} + \lambda v_{js} - \beta_{js} = 0 \quad \forall j \in \{1, \dots, d\}, s \in \{1, \dots, k\} \quad (8.9)$$

We would like to eliminate the Lagrangian parameters and set up the optimization conditions purely in terms of U and V . In this context, the complementary slackness components of the Kuhn-Tucker optimality conditions turn out to be very helpful. These conditions are $u_{is}\alpha_{is} = 0$ and $v_{js}\beta_{js} = 0$ over all parameters. By multiplying Equation 8.8 with u_{is} and multiplying Equation 8.9 with v_{js} , one obtains a condition purely in terms of the primal variables:

$$-(DV)_{is}u_{is} + (UV^T V)_{is}u_{is} + \lambda u_{is}^2 - \underbrace{\alpha_{is}u_{is}}_0 = 0 \quad \forall i \in \{1, \dots, n\}, s \in \{1, \dots, k\} \quad (8.10)$$

$$-(D^T U)_{js}v_{js} + (V U^T U)_{js}v_{js} + \lambda v_{js}^2 - \underbrace{\beta_{js}v_{js}}_0 = 0 \quad \forall j \in \{1, \dots, d\}, s \in \{1, \dots, k\} \quad (8.11)$$

One can rewrite these optimality conditions, so that a single parameter occurs on one side of the condition:

$$u_{is} = \frac{[(DV)_{is} - \lambda u_{is}]u_{is}}{(UV^T V)_{is}} \quad \forall i \in \{1, \dots, n\}, s \in \{1, \dots, k\} \quad (8.12)$$

$$v_{js} = \frac{[(D^T U)_{js} - \lambda v_{js}]v_{js}}{(V U^T U)_{js}} \quad \forall j \in \{1, \dots, d\}, s \in \{1, \dots, k\} \quad (8.13)$$

The aforementioned conditions can be used in order to perform iterative updates. A small value of ϵ is typically added to the denominator to avoid ill conditioning. Therefore, the iterative approach starts by initializing the parameters in U and V to nonnegative random values in $(0, 1)$ and then uses the following updates:

$$u_{is} \leftarrow \frac{[(DV)_{is} - \lambda u_{is}]u_{is}}{(UV^T V)_{is} + \epsilon} \quad \forall i \in \{1, \dots, n\}, s \in \{1, \dots, k\} \quad (8.14)$$

$$v_{js} \leftarrow \frac{[(D^T U)_{js} - \lambda v_{js}]v_{js}}{(V U^T U)_{js} + \epsilon} \quad \forall j \in \{1, \dots, d\}, s \in \{1, \dots, k\} \quad (8.15)$$

These iterations are then repeated to convergence. Improved initialization provides significant advantages [76].

As in all other forms of matrix factorization, it is possible to convert the factorization UV^T into the three-way factorization $Q\Sigma P^T$ by using the approach discussed in Section 7.2.7 of Chapter 7. For a nonnegative factorization, it makes sense to use L_1 -normalization on each column of U and V , so that the columns of the resulting matrices Q and P each sum to 1. This type of normalization makes nonnegative factorization similar to

a closely related factorization known as *Probabilistic Semantic Analysis (PLSA)*. The main difference between PLSA and nonnegative matrix factorization is that the former uses a maximum likelihood optimization function (or *I-divergence* objective) whereas nonnegative matrix factorization (typically) uses the Frobenius norm. Refer to Section 8.4.5.

8.4.3 Interpretability of Nonnegative Matrix Factorization

Nonnegative matrix factorization is a highly interpretable form of decomposition. In order to understand this point, we will consider the case in which the matrix D is an $n \times d$ document-term matrix; correspondingly, the corpus contains n documents and d words. The (i, j) th entry is the frequency of term j in document i . The $n \times k$ matrix U can be viewed as the factors of documents, whereas the $d \times k$ matrix V can be viewed as the factors of words. Each of the k factors can be viewed as a *topic* or a cluster of related documents. The r th columns U_r and V_r of each of U and V respectively contain document- and word-membership information about the r th topic (or cluster) in the data. The n entries in U_r correspond to the nonnegative components (coordinates) of the n documents along the r th topic. If a document strongly belongs to topic r , then it will have a very positive coordinate in U_r . Otherwise, its coordinate will be zero or mildly positive (representing noise). Similarly, the r th column V_r of V provides the frequent vocabulary of the r th cluster. Terms that are highly related to a particular topic will have large components in V_r . The k -dimensional representation of each document is provided by the corresponding row of U . This approach allows a document to belong to multiple clusters, because a given row in U might have multiple positive coordinates. For example, if a document discusses both science and history, it will have components along latent components with science-related and history-related vocabularies. This provides a more realistic “sum-of-parts” decomposition of the corpus along various topics, which is primarily enabled by the nonnegativity of U and V . In fact, one can create a decomposition of the document-term matrix into k different rank-1 document-term matrices corresponding to the k topics captured by the decomposition. Let us treat U_r as an $n \times 1$ matrix and V_r as a $d \times 1$ matrix. If the r th component is related to science, then $U_r V_r^T$ is an $n \times d$ document-term matrix containing the science-related portion of the original corpus. Then the decomposition of the document-term matrix is defined as the sum of the following components:

$$D \approx \sum_{r=1}^k U_r V_r^T \quad (8.16)$$

This decomposition represents matrix multiplication as a sum of outer-products; in this particular case, the nonnegativity of each additive component $U_r V_r^T$ makes it interpretable as a “document-term matrix.”

8.4.4 Example of Nonnegative Matrix Factorization

In order to illustrate the semantic interpretability of nonnegative matrix factorization, let us revisit the same example used in Section 7.2.4.3 of Chapter 7, and create a decomposition in terms of nonnegative matrix factorization:

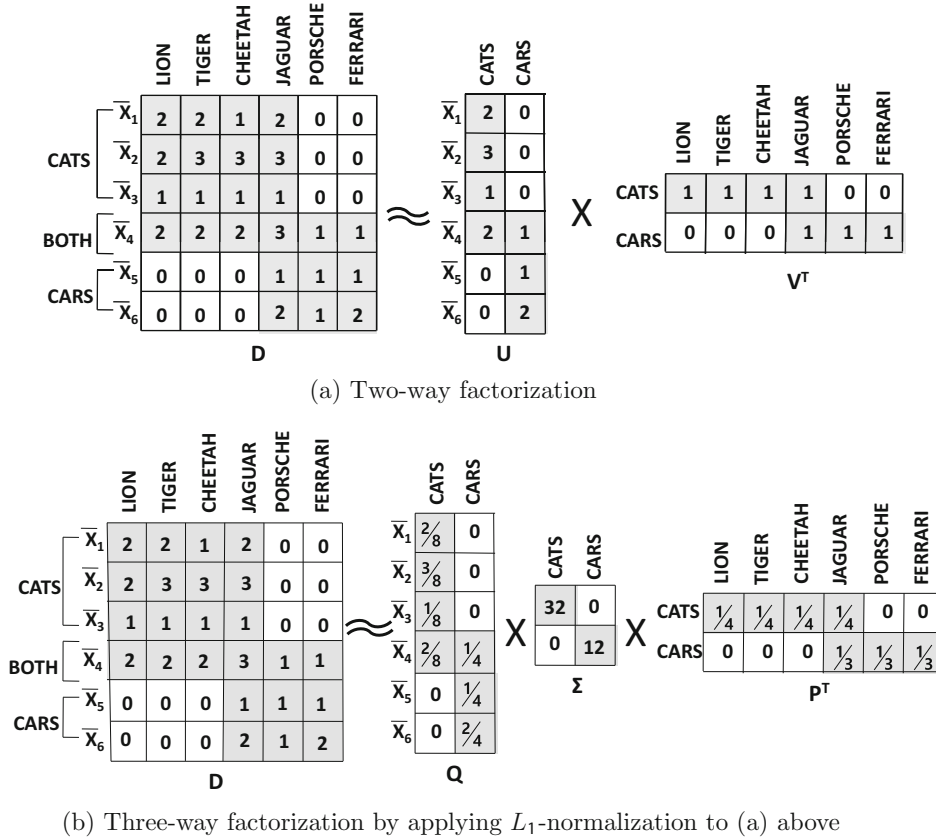


Figure 8.3: The highly interpretable decomposition of nonnegative matrix factorization

$$D = \begin{pmatrix} & \text{lion} & \text{tiger} & \text{cheetah} & \text{jaguar} & \text{porsche} & \text{ferrari} \\ \text{Document-1} & 2 & 2 & 1 & 2 & 0 & 0 \\ \text{Document-2} & 2 & 3 & 3 & 3 & 0 & 0 \\ \text{Document-3} & 1 & 1 & 1 & 1 & 0 & 0 \\ \text{Document-4} & 2 & 2 & 2 & 3 & 1 & 1 \\ \text{Document-5} & 0 & 0 & 0 & 1 & 1 & 1 \\ \text{Document-6} & 0 & 0 & 0 & 2 & 1 & 2 \end{pmatrix}$$

This matrix represents topics related to both cars and cats. The first three documents are related to cats, the fourth is related to both, and the last two are related to cars. The *polysemous* word “jaguar” is present in documents of both topics.

A highly interpretable nonnegative factorization of rank-2 is shown in Figure 8.3(a). We have shown an approximate decomposition containing only integers for simplicity, although the optimal solution would (almost always) be dominated by floating point numbers in practice. It is clear that the first latent concept is related to cats and the second latent concept is related to cars. Furthermore, documents are represented by two non-negative coordinates indicating their affinity to the two topics. Correspondingly, the first three documents have strong positive coordinates for cats, the fourth has strong positive coordinates in both, and

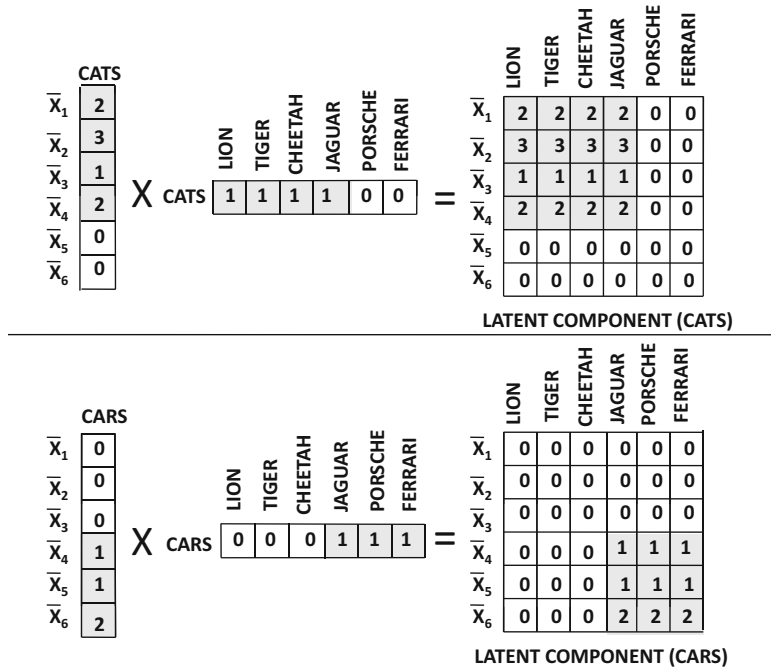


Figure 8.4: The highly interpretable “sum-of-parts” decomposition of the document-term matrix into rank-1 matrices representing different topics

the last two belong only to cars. The matrix V tells us that the vocabularies of the various topics are as follows:

- Cats:** lion, tiger, cheetah, jaguar
- Cars:** jaguar, porsche, ferrari

It is noteworthy that the polysemous word “jaguar” is included in the vocabulary of both topics, and its usage is automatically inferred from its context (i.e., other words in document) during the factorization process. This fact becomes especially evident when we decompose the original matrix into two rank-1 matrices according to Equation 8.16. This decomposition is shown in Figure 8.4 in which the rank-1 matrices for cats and cars are shown. It is particularly interesting that the occurrences of the polysemous word “jaguar” are nicely divided up into the two topics, which roughly correspond with their usage in these topics.

As discussed in Section 7.2.7 of Chapter 7, any two-way matrix factorization can be converted into a standardized three-way factorization. In the case of nonnegative matrix factorization, it is common to use L_1 -normalization, rather than L_2 -normalization (which is used in SVD). The three-way normalized representation is shown in Figure 8.3(b), and it tells us a little bit more about the relative frequencies of the two topics. Since the diagonal entry in Σ is 32 for cats in comparison with 12 for cars, it indicates that the topic of cats is more dominant than cars. This is consistent with the observation that more documents and terms in the collection are associated with cats as compared to cars.

8.4.5 The I-Divergence Objective Function

The previous section used the Frobenius norm in order to ensure that D is as close to UV^T as possible. However, it is possible to use a different objective function, referred to as the I-divergence function, in order to achieve the same goal [79]. This objective function is formulated as follows:

$$\begin{aligned} & \text{Minimize}_{U,V} \sum_{i=1}^n \sum_{j=1}^d \left(D_{ij} \log \left\{ \frac{D_{ij}}{(UV^T)_{ij}} \right\} - D_{ij} + (UV^T)_{ij} \right) \\ & \text{subject to:} \\ & U \geq 0, \quad V \geq 0 \end{aligned}$$

This formulation takes on its minimum value $D = UV^T$. The reader is advised to solve the following problem to obtain more insight on this point:

Problem 8.4.1 Consider the following function $F(x)$:

$$F(x) = a \cdot \log(a/x) - a + x$$

Here, a is a constant. Show that the function achieves its minimum value $x = a$.

In nonnegative matrix factorization, the function $F(x)$ is applied to each reconstructed entry x and (corresponding) observed entry a , and then this value is aggregated over all matrix entries. In the case of the Frobenius norm, the function $\|x - a\|^2$ is used instead of $F(x)$. In both cases, the objective function tries to make x as close to a as possible. The model requires the following iterative solution for $U = [u_{is}]$ and $V = [v_{js}]$:

$$\begin{aligned} u_{is} & \leftarrow u_{is} \frac{\sum_{j=1}^d [D_{ij} v_{js} / (UV^T)_{ij}]}{\sum_{j=1}^d v_{js}} & \forall i, s \\ v_{js} & \leftarrow v_{js} \frac{\sum_{i=1}^n [D_{ij} u_{is} / (UV^T)_{ij}]}{\sum_{i=1}^n u_{is}} & \forall j, s \end{aligned}$$

The two-way factorization can be converted into a normalized three-way factorization using the approach discussed in Section 7.2.7 of Chapter 7. The three-way factorization can be interpreted from the perspective of a *probabilistic generative model*, which is *identical to probabilistic latent semantic analysis*.

8.5 Weighted Matrix Factorization

In weighted matrix factorization, weights are associated with the individual entries in the matrix, because errors on some entries are considered more important than others. The optimization model is similar to the unconstrained matrix factorization model discussed in Section 8.3. Furthermore, the application of matrix factorization to incomplete data (cf. Section 8.3.2) is a relatively trivial special case of weighted matrix factorization; in this case, the weights of observed entries are set to 1 and the weights of missing entries are set to 0. However, the most important (and somewhat nontrivial) use case, which will be discussed in this section, turns out to be an alternative to nonnegative matrix factorization for sparse matrices containing frequency counts. A surprising number of real-world applications, such as implicit feedback data and graph adjacency matrices, belong to this category.

In weighted matrix factorization, we have a weight w_{ij} associated with the (i, j) th entry of the $n \times d$ matrix $D = [x_{ij}]$ to be factorized. As in the case of unconstrained matrix factorization, we assume that the two factor matrices are the $n \times k$ matrix U and the $d \times k$ matrix V . Then, the objective function of weighted matrix factorization is as follows:

$$\text{Minimize } J = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^d w_{ij} \left(x_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right)^2 + \frac{\lambda}{2} \sum_{i=1}^n \sum_{s=1}^k u_{is}^2 + \frac{\lambda}{2} \sum_{j=1}^d \sum_{s=1}^k v_{js}^2$$

Note that this objective function is different from that of unconstrained matrix factorization only in terms of the weights w_{ij} of the entries. The partial derivative of the objective function with respect to the various parameters can be expressed in terms of the error $e_{ij} = x_{ij} - \hat{x}_{ij}$ of the factorization as follows:

$$\begin{aligned} \frac{\partial J}{\partial u_{iq}} &= \sum_{i=1}^n \sum_{j=1}^d (w_{ij} e_{ij})(-v_{jq}) + \lambda u_{iq} \quad \forall i \in \{1 \dots n\}, q \in \{1 \dots k\} \\ \frac{\partial J}{\partial v_{jq}} &= \sum_{i=1}^n \sum_{j=1}^d (w_{ij} e_{ij})(-u_{iq}) + \lambda v_{jq} \quad \forall j \in \{1 \dots d\}, q \in \{1 \dots k\} \end{aligned}$$

The main difference from unconstrained matrix factorization is in terms of weighting the errors with w_{ij} . In order to express the aforementioned derivatives in matrix form, we define E as an $n \times d$ error matrix in which the (i, j) th entry is e_{ij} . Furthermore $W = [w_{ij}]$ is an $n \times d$ matrix containing the weights of various entries.

$$\begin{aligned} \frac{\partial J}{\partial U} &= -(W \odot E)V + \lambda U \\ \frac{\partial J}{\partial V} &= -(W \odot E)^T U + \lambda V \end{aligned}$$

Here, the notation $\odot$ indicates elementwise multiplication between two matrices of exactly the same size. The weight matrix W essentially controls the importance of the errors of the individual entries in gradient descent. One can, therefore, express the gradient descent updates for the matrices U and V as follows:

$$\begin{aligned} U &\Leftarrow U - \alpha \frac{\partial J}{\partial U} = U(1 - \alpha\lambda) + \alpha(W \odot E)V \\ V &\Leftarrow V - \alpha \frac{\partial J}{\partial V} = V(1 - \alpha\lambda) + \alpha(W \odot E)^T U \end{aligned}$$

Here, $\alpha > 0$ is the learning rate.

8.5.1 Practical Use Cases of Nonnegative and Sparse Matrices

Interestingly, weighted matrix factorization is often used in nonnegative and sparse matrices like implicit feedback data, graph adjacency matrices, and various text-centric matrices. In these cases, the weight matrix W and factorized matrix D are both defined as functions of the original raw *quantity* matrix Q . The matrix Q might correspond to the number of occurrences of customer buying behavior, link-link interactions, or document-word interactions. Although it is possible to use $D = Q$, the common approach is to modify the entries in Q in some way (such as setting them to 0-1 values, depending on whether they are non-zero or

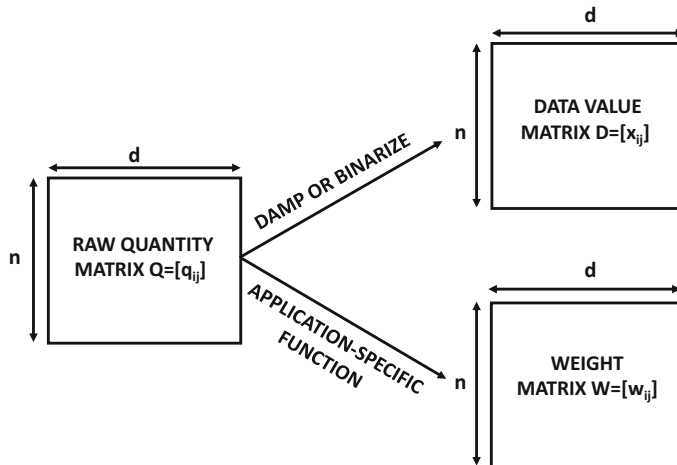


Figure 8.5: Deriving the data value and weight matrices from raw quantity matrix

not). Setting the entries to binary values makes sense in cases where the final prediction of the factorization is intended to be binary (e.g., recommend an item or link). In such a case, a new *binary* data matrix D is used in lieu of Q , where the (i, j) th entry x_{ij} of D is set to 1 when the corresponding entry of $Q = [q_{ij}]$ is non-zero. In other applications, the values of the *raw* data matrix Q are “damped” before factorization. In other words, each raw entry q_{ij} is replaced with the damped value $x_{ij} = f(q_{ij})$, where $f(\cdot)$ is a damping function like the square-root or logarithm. An example of such an approach is the GloVe embedding [101] for factorization of matrices derived from text (cf. Section 8.5.5). The weight matrix W is also derived as a function of the quantity matrix. This overall process is illustrated in Figure 8.5.

The choice of the weight matrix $W = [w_{ij}]$ is, however, more application-specific. In some cases, the weight matrix is set to the entries in D when they are non-zero. However, the zero entries also need to be set to specific weights. Typically, the weight of a zero entry is either set to a constant value or it is set to a column-specific value. Allowing non-zero weights on zero entries amounts to using *negative sampling* in the context of stochastic gradient descent. As we will see later, this type of negative sampling is important in most applications. While the weight matrix is technically dense because of the non-zero weights on zero entries, it can still be represented in compressed form. This is because all zero entries in a column have the same weight, and therefore one only needs to store the column-specific weight.

Why is this type of weighted matrix factorization more desirable than vanilla nonnegative matrix factorization? The reason is that the data matrix Q is sparse, and the vast majority of entries are 0s. In such cases, the fact that an entry is non-zero is more important than the specific magnitude of that value. Factorizing the values of the matrix Q might sometimes cause problems when the different entries of the matrix vary by orders of magnitude. This type of situation can occur in word matrices with large variations in word frequencies, or in graphs with power-law frequency distributions. If one simply performs value-based factorization, the preponderance of the (relatively unimportant) zero entries and the large magnitudes of a very small number of entries might play too large a role in the factorization. As a result, the modeling of most of the important entries in the matrix will be poor. As

discussed in [65] in the case of ratings matrices, the general principle for treating raw numerical values is as follows:

“The numerical value of explicit feedback [value in a dense matrix] indicates preference, whereas the numerical value of implicit feedback [value in a sparse matrix] indicates confidence.”

Of course, a zero value in a sparse matrix does not necessarily indicate zero confidence, which is why one must resort to setting some non-zero weights on default values. In this section, we will provide several application-specific examples of scenarios in which sparse values should be treated as weights. Another useful property of weighted matrix factorization is that it allows a very efficient trick for parameter learning when most of the entries are zeros.

8.5.2 Stochastic Gradient Descent

The stochastic gradient descent procedure of Section 8.3.2.1 samples each entry of the matrix with equal probability in order to perform the updates. In weighted matrix factorization, the entries are sampled with probabilities that are proportional to their weights. In the (deterministic) gradient descent updates discussed earlier in this section, the weight of each entry is explicitly multiplied with the error of each entry in the update. Stochastic gradient descent replaces explicit weighting with weighted sampling, while keeping the form of the update unchanged from the unweighted case. Aside from this difference in how error entries are sampled, the algebraic form of the updates remains unchanged from the unweighted case:

$$\begin{aligned}\bar{u}_i &\leftarrow \bar{u}_i(1 - \alpha\lambda) + e_{ij}\bar{v}_j \\ \bar{v}_j &\leftarrow \bar{v}_j(1 - \alpha\lambda) + e_{ij}\bar{u}_i\end{aligned}$$

Here, $\alpha > 0$ is the learning rate. Here, $\bar{u}_i$ represents the i th row of the $n \times k$ matrix U and $\bar{v}_j$ represents the j th row of the $d \times k$ matrix V . Note that exactly $2k$ entries in the matrices U and V are updated for each entry in the matrix.

This type of weighted matrix factorization is particularly efficient when the vast majority of the (raw) entries in the matrix are zeros. In many applications, the number of entries in the $n \times d$ matrix D might be very large, but the number of non-zero entries is several orders of magnitude lower. This is common in applications such as graphs in which a $10^6 \times 10^6$ adjacency matrix might have only 10 non-zero entries in each row. Therefore, the weight matrix is also sparse, and one only needs to keep track of positive sampling probabilities (i.e., non-zero entries in the original matrix D). All the zero weights are aggregated into a single negative sampling probability. The stochastic gradient descent procedure works as follows:

1. A coin is tossed with probability equal to the negative sampling rate. If the coin toss is a success, a random entry is treated as a negative entry, and an update is performed with the random entry (assuming that the observed value of the random entry is zero).
2. If the coin toss in the previous step is a failure, then a positive entry is sampled with probability proportional to its weight. Subsequently, stochastic gradient descent is performed with the positive entry.

This process of randomly sampling entries in proportion to their weights is iterated to convergence.

8.5.2.1 Why Negative Sampling Is Important

It is noteworthy that using negative sampling is particularly important, especially in cases where there is not much variation among the positive entries. For example, if one applied this procedure to a sparse binary matrix while setting the negative sampling rate to 0, one possible “optimal” solution is to obtain U and V as matrices in which each entry is $1/\sqrt{k}$. It is not difficult to see that UV^T will be a matrix of 1s, in which there are no errors on the positive entries and drastic errors on the negative entries. A negative sampling rate of 0 implicitly defines the objective function only over the positive entries, and therefore an optimal solution of zero error can be reached even when the quality of predictions is poor. One can view this situation as a type of overfitting. *In general, the weights w_{ij} of zero entries in sparse settings should always be set to non-zero values.*

8.5.3 Application: Recommendations with Implicit Feedback Data

Consider a case in which the original data set contains the amount that the user i bought for item j . Let $Q = [q_{ij}]$ be the original matrix of quantities bought by the user. In such a case, we create an $n \times d$ data matrix $D = [x_{ij}]$ as the binary indicator matrix of the quantities that were bought by the user. Therefore, the entries of the matrix D are defined as follows:

$$x_{ij} = \begin{cases} 1 & q_{ij} > 0 \\ 0 & q_{ij} = 0 \end{cases}$$

Therefore, the $n \times d$ matrix D is now a binary indicator matrix. Furthermore, the work in [65] suggests the following heuristic to select the weight w_{ij} of entry (i, j) :

$$w_{ij} = 1 + \theta \cdot q_{ij} \quad (8.17)$$

The above weighting scheme ensures that *zero entries in the raw matrix Q have a non-zero weight*. These non-zero weights of zero entries define the negative sampling probability, when added over the various zero entries. It is suggested in [65] to use a value of $\theta = 40$. Then, the weighted matrix factorization of D is used to find the factor matrices U and V . The entries (i, j) with large values of $(UV^T)_{ij}$ are suggested recommendations of item j for user i .

8.5.4 Application: Link Prediction in Adjacency Matrices

Link prediction is a recommendation problem in every sense of the word. Instead of user-item matrices, we have the $n \times n$ node-node adjacency matrix $Q = [q_{ij}]$. The value q_{ij} of the entry (i, j) is the weight of edge (i, j) . Such graphs are sparse, and therefore most values of q_{ij} are 0s. For example, in a bibliographic network, this weight might correspond to the number of publications between authors i and j . As in the case of the user-item recommendation application, the *binarized* version of the raw $n \times n$ node-node matrix is defined as follows:

$$x_{ij} = \begin{cases} 1 & q_{ij} > 0 \\ 0 & q_{ij} = 0 \end{cases}$$

The setting of the weight matrix requires additional thought, compared to the user-item recommendation application. One issue is that the weights of the edges in these graphs have a very high level of variation. Therefore, if care is not taken in selecting the weights,

a few weights can dominate the factorization. This is obviously undesirable. Therefore, one possibility is to define w_{ij} with logarithmic damping:

$$w_{ij} = 1 + \theta \cdot \log(1 + q_{ij})$$

The value of θ can be tuned by testing its accuracy over a set of entries that are excluded from the sampling process in stochastic gradient descent.

8.5.5 Application: Word-Word Context Embedding with GloVe

The acronym GloVe stands for Global Vectors for Word Representation. The goal is to create multidimensional embeddings of words based on the other words in their context windows. Therefore, words that are similar in terms of the distribution of words in the window-based locality of these words will tend to have similar embeddings. The matrix Q in GloVe is a $d \times d$ word-context matrix. The (i, j) th entry in the matrix is the number of times that the word j occurs within a pre-defined distance δ of the word i in sentences of the document. The value of δ is typically a small quantity such as 4 or 5. The rows of the factor matrices U and V can be concatenated (or even added) to create the embeddings of the individual words. Note that there are exactly as many rows of these matrices as the number of words. These types of embeddings tend to have greater linguistic and semantic significance than those obtained from methods like latent semantic analysis.

The matrix Q is then damped to create the data matrix $D = [x_{ij}]$, which contains the values to be factorized:

$$x_{ij} = \log(1 + q_{ij})$$

Note that a binary data matrix D is no longer used in this application. The weight w_{ij} is defined as follows:

$$w_{ij} = \begin{cases} \min \left\{ 1, \frac{c_{ij}}{M} \right\}^\alpha & q_{ij} > 0 \\ 0 & q_{ij} = 0 \end{cases}$$

The values of M and α are recommended to be 100 and $3/4$, respectively, based on empirical considerations. It is possible to enhance this basic model in a number of ways, such as by the use of bias variables.

Note that GloVe sets the negative sampling probability to zero, and therefore depends almost entirely on the variation among the different *non-zero* values of x_{ij} . This is an unusual and controversial design choice, and it is very different from almost all other known techniques for weighted matrix factorization. It is significant that GloVe does not try to extract binary values of x_{ij} from the original quantity matrix $Q = q_{ij}$. Trying to use binary values of x_{ij} would be disastrous in the case of GloVe (cf. Exercise 9). A directly competing method, referred to as *word2vec*, plays great emphasis on negative sampling in order to achieve high-quality results [91, 92]. If the values of q_{ij} do not vary significantly in a given collection, it is possible for GloVe to provide overfitted results (cf. Section 8.5.2.1). This does not seem to be the case in practice, as GloVe seems to provide reasonably good results (based on independent evaluations by researchers and practitioners). This might possibly be a result of the fact that there is sufficient variation among the non-zero frequency counts (even after damping) in word-word context matrices. This property might not be the case in other domains, and therefore one should generally be cautious of factorizations that do not use some type of negative sampling.

8.6 Nonlinear Matrix Factorizations

All the models discussed so far in this chapter factorize a matrix as $D \approx UV^T$, which is linear in both U and V . Logistic and maximum margin matrix factorizations deviate from this linearity by using an elementwise function of UV^T to derive D . Although these methods are naturally designed for binary and sparse matrices, the general principle of nonlinearity can be extended to real-valued matrices with the appropriate choice of prediction function.

For the binary matrices in this section, the (i, j) th binary value is associated with a weight w_{ij} . Such factorizations are also designed for sparse matrices, where the values of the entries are treated as weights. These types of factorizations can be used for any of the applications that are used in conjunction with weighted matrix factorization. For example, all the applications for recommendations, link prediction, and text processing (discussed in Sections 8.5.3–8.5.5) can be directly supported by logistic and maximum-margin matrix factorization.

8.6.1 Logistic Matrix Factorization

Unlike weighted matrix factorization, logistic matrix factorization uses a nonlinear prediction function that can be interpreted as a probabilistic model. Let U and V be the $n \times k$ and $d \times k$ factor matrices for the original $n \times d$ matrix $Q = [q_{ij}]$ of sparse frequency counts. Logistic matrix factorization applies the logistic sigmoid function $F(x)$ to each entry of UV^T in order to generate a probability matrix P :

$$P = F(UV^T)$$

Here, the function $F(\cdot)$ is applied in an entry-wise fashion, and is defined as follows:

$$F(x) = \frac{1}{1 + \exp(-x)}$$

Each entry of the $n \times d$ matrix P is a probability value drawn from $(0, 1)$, and the goal is to maximize the log-likelihood of the observed data matrix based on these probabilities. As in the case of weighted matrix factorization, we create the binarized matrix $D = [x_{ij}]$ as follows:

$$x_{ij} = \begin{cases} 1 & q_{ij} > 0 \\ 0 & q_{ij} = 0 \end{cases}$$

We utilize a user-driven parameter m , which is the ratio of the *aggregate* weight of the negative entries to the aggregate weight of the positive entries. The weight w_{ij} of each entry of the matrix is defined as follows:

$$w_{ij} = \begin{cases} q_{ij} & q_{ij} > 0 \\ m(\sum_{s=1}^d q_{is})/d & q_{ij} = 0 \end{cases}$$

The value of m is set in a domain-specific way, and it is often a small integer such as 5. Under the assumption that the matrix is sparse, the sum of the negative entries in each row is roughly $m(\sum_{s=1}^d q_{is})$. Therefore, the weights of the negative entries are m times the weights of the positive entries, where m is a user-driven parameter. Implicitly, the negative entries are underweighted by this approach, because a sparse matrix will often contain negative entries that are hundreds of times the number of positive entries, whereas m is a small value such as 5.

A key point in logistic matrix factorization is what we would like the (learned) probability matrix $P = p_{ij}$ to have a large value of p_{ij} when x_{ij} is 1, and a small value of p_{ij} when x_{ij} is 0. This can be achieved with a log-likelihood objective function, which is defined as follows:

$$J = - \sum_{i=1}^n \sum_{j=1}^d w_{ij} [x_{ij} \log(p_{ij}) + (1 - x_{ij}) \log(1 - p_{ij})]$$

Here, it is evident that this loss function is always nonnegative, and it takes² on its minimum value of 0 when $p_{ij} = x_{ij}$. Recall that each p_{ij} is the (i, j) th entry of $F(UV^T)$, and it is defined as follows:

$$p_{ij} = \frac{1}{1 + \exp(-\bar{u}_i \cdot \bar{v}_j)}$$

Here, $\bar{u}_i$ is the i th row of the $n \times k$ matrix U , and $\bar{v}_j$ is the j th row of the $d \times k$ matrix V . Therefore, one can substitute this value of p_{ij} in the objective function in order to obtain the following loss function for logistic matrix factorization:

$$J = - \sum_{i=1}^n \sum_{j=1}^d w_{ij} \left[x_{ij} \log \left(\frac{1}{1 + \exp(-\bar{u}_i \cdot \bar{v}_j)} \right) + (1 - x_{ij}) \log \left(\frac{1}{1 + \exp(\bar{u}_i \cdot \bar{v}_j)} \right) \right]$$

Now that we have set up the objective function of logistic matrix factorization, it remains to derive the gradient descent steps.

8.6.1.1 Gradient Descent Steps for Logistic Matrix Factorization

In order to perform the gradient-descent updates, we need to compute the gradient of the objective function with respect to the k -dimensional vectors $\bar{u}_i$ and $\bar{v}_j$. This is best achieved using the chain rule in matrix calculus:

$$\begin{aligned} \frac{\partial J}{\partial \bar{u}_i} &= \sum_{j=1}^d \frac{\partial J}{\partial (\bar{u}_i \cdot \bar{v}_j)} \frac{\partial (\bar{u}_i \cdot \bar{v}_j)}{\partial \bar{u}_i} = \sum_{j=1}^d \frac{\partial J}{\partial (\bar{u}_i \cdot \bar{v}_j)} \bar{v}_j \\ \frac{\partial J}{\partial \bar{v}_j} &= \sum_{i=1}^n \frac{\partial J}{\partial (\bar{u}_i \cdot \bar{v}_j)} \frac{\partial (\bar{u}_i \cdot \bar{v}_j)}{\partial \bar{v}_j} = \sum_{i=1}^n \frac{\partial J}{\partial (\bar{u}_i \cdot \bar{v}_j)} \bar{u}_i \end{aligned}$$

Note that the partial derivative of $\bar{u}_i \cdot \bar{v}_j$ with respect to either $\bar{u}_i$ or $\bar{v}_j$ is obtained using identity (v) of Table 4.2(a). It is relatively easy to compute the partial derivative of J with respect to $\bar{u}_i \cdot \bar{v}_j$ because the objective function is defined as a function of this quantity. By computing this derivative and substituting in the above equations, we obtain the following:

$$\begin{aligned} \frac{\partial J}{\partial \bar{u}_i} &= - \sum_{j=1}^d \frac{w_{ij} x_{ij} \bar{v}_j}{1 + \exp(\bar{u}_i \cdot \bar{v}_j)} + \sum_{j=1}^d \frac{w_{ij} (1 - x_{ij}) \bar{v}_j}{1 + \exp(-\bar{u}_i \cdot \bar{v}_j)} \\ \frac{\partial J}{\partial \bar{v}_j} &= - \sum_{i=1}^n \frac{w_{ij} x_{ij} \bar{u}_i}{1 + \exp(\bar{u}_i \cdot \bar{v}_j)} + \sum_{i=1}^n \frac{w_{ij} (1 - x_{ij}) \bar{u}_i}{1 + \exp(-\bar{u}_i \cdot \bar{v}_j)} \end{aligned}$$

²Strictly speaking, the objective function is not defined when p_{ij} is 0 or 1. However, the loss is zero when $p_{ij} \rightarrow x_{ij}$ in the limit. A logistic function will never yield values of exactly 0 or 1 for p_{ij} .

With these derivatives, a straightforward gradient-descent procedure can be applied at learning rate $\alpha > 0$:

$$\begin{aligned}\bar{u}_i &\leftarrow \bar{u}_i - \alpha \frac{\partial J}{\partial \bar{u}_i} \quad \forall i \\ \bar{v}_j &\leftarrow \bar{v}_j - \alpha \frac{\partial J}{\partial \bar{v}_j} \quad \forall j\end{aligned}$$

The gradient-descent approach computes the exact derivative of the objective function. What about stochastic gradient descent? It turns out that mini-batch stochastic gradient descent is particularly popular in the case of logistic matrix factorization. Since the total weight of negative entries is m times the number of positive entries, it is particularly common to sample one positive entry together with m negative entries for stochastic gradient descent. Interestingly, when this type of stochastic gradient descent is used for updates and it is applied to word-word context matrices with weight set to the frequencies of contexts, the resulting approach is *identical* to the backpropagation-based updates of the *word2vec* algorithm. The weights of the negative entries in the i th row is $K_i(\sum_{s=1}^d q_{is})^\alpha$, where $\alpha = 3/4$. The value of K_i is chosen to ensure that the weights of the negative entries in the i th row sum to roughly $m(\sum_{s=1}^d q_{is})$. Therefore, *word2vec* is simply an instantiation of logistic matrix factorization, and it provides an alternative to the GloVe algorithm discussed earlier in this chapter.

Problem 8.6.1 (Regularized Logistic Matrix Factorization) *Write an objective function for logistic matrix factorization that uses L_2 -regularization. Derive the gradient-descent steps.*

8.6.2 Maximum Margin Matrix Factorization

Just as logistic regression is closely related to support vector machines (cf. Figure 4.9 of Chapter 4), logistic matrix factorization is closely related to maximum margin matrix factorization. The following exposition is roughly based on [120], although we simplify the algorithm in many respects while allowing the use of weights. As in the case of logistic matrix factorization, a *binary* $n \times d$ data matrix $D = [x_{ij}]$ and an $n \times d$ weight matrix $W = [w_{ij}]$ is derived from the raw quantity matrix $Q = [q_{ij}]$. The original work in [120] does not discuss the use of weights. In practice, methods like logistic matrix factorization and maximum-margin matrix factorization are most useful when weights are allowed. This is because many sparse matrices are associated with small counts on the non-zero entries (in real settings of implicit feedback). Furthermore, while the work in [120] provides a dual learning algorithm, we provide a much simpler primal algorithm.

Note that the 0-1 entry x_{ij} from the data matrix D can be converted to a value y_{ij} drawn from $\{-1, +1\}$ by using the following transformation:

$$y_{ij} = 2x_{ij} - 1$$

One can also introduce the matrix $Y = [y_{ij}]$, which is defined as $Y = 2D - \bar{\mathbf{1}}_n \bar{\mathbf{1}}_d^T$. Here, $\bar{\mathbf{1}}_k$ is a column vector of k ones, and therefore $\bar{\mathbf{1}}_n \bar{\mathbf{1}}_d^T$ is an $n \times d$ matrix of 1s. We choose to use y_{ij} instead of x_{ij} in the objective function below because it is easier to show similarity with the SVM objective function by doing so. Let U and V be the $n \times k$ and $d \times k$ factor matrices, respectively. The i th rows of U and V are denoted by $\bar{u}_i$ and $\bar{v}_i$ respectively. Then, the predicted value of the (i, j) th entry, denoted by $\hat{y}_{ij}$, is defined as follows:

$$\hat{y}_{ij} = \bar{u}_i \cdot \bar{v}_j$$

Unlike logistic matrix factorization, the predicted value $\hat{y}_{ij}$ is intended to match a quantity y_{ij} from $\{-1, +1\}$, rather than a value from $\{0, 1\}$. An important point here is that entries with large absolute values of $\hat{y}_{ij}$ are not penalized, as long as their sign is correct. This is because this factorization predicts the original entries by using the sign function on UV^T instead of using the absolute deviation from observed values:

$$Y \approx \text{sign}(UV^T)$$

This type of approach is exactly analogous to the prediction approach in an SVM. As in the case of SVMs, the hinge loss is used on the individual entries:

$$\text{Hinge}(i, j) = \max\{0, 1 - y_{ij}\hat{y}_{ij}\} = \max\{0, 1 - y_{ij}[\bar{u}_i \cdot \bar{v}_j]\}$$

This is a margin-based objective function, because an entry is not penalized only when its predicted value matches the sign of the original binary value with a sufficient margin of 1. Then, the overall objective function of maximum margin factorization (without regularization) can be expressed as follows:

$$J = \sum_{i=1}^n \sum_{j=1}^d w_{ij} \max\{0, 1 - y_{ij}[\bar{u}_i \cdot \bar{v}_j]\}$$

As in the case of logistic matrix factorization, one can use the chain rule to compute the derivative.

$$\begin{aligned} \frac{\partial J}{\partial \bar{u}_i} &= \sum_{j=1}^d \frac{\partial J}{\partial (\bar{u}_i \cdot \bar{v}_j)} \bar{v}_j = - \sum_{j: y_{ij}(\bar{u}_i \cdot \bar{v}_j) < 1} w_{ij} y_{ij} \bar{v}_j \\ \frac{\partial J}{\partial \bar{v}_j} &= \sum_{i=1}^n \frac{\partial J}{\partial (\bar{u}_i \cdot \bar{v}_j)} \bar{u}_i = - \sum_{i: y_{ij}(\bar{u}_i \cdot \bar{v}_j) < 1} w_{ij} y_{ij} \bar{u}_i \end{aligned}$$

It is common to use L_2 -regularization, in which case the gradients above are adjusted by $\lambda \bar{u}_i$ and $\lambda \bar{v}_j$, respectively. Here, $\lambda > 0$ is the regularization parameter. Therefore, at learning rate $\alpha > 0$, the gradient-descent updates of maximum-margin matrix factorization are as follows:

$$\begin{aligned} \bar{u}_i &\leftarrow \bar{u}_i(1 - \alpha\lambda) + \alpha \sum_{j: y_{ij}(\bar{u}_i \cdot \bar{v}_j) < 1} w_{ij} y_{ij} \bar{v}_j \quad \forall i \\ \bar{v}_j &\leftarrow \bar{v}_j(1 - \alpha\lambda) + \alpha \sum_{i: y_{ij}(\bar{u}_i \cdot \bar{v}_j) < 1} w_{ij} y_{ij} \bar{u}_i \quad \forall j \end{aligned}$$

Just as SVMs and logistic regression provide very similar results for classification of binary labels, logistic matrix factorization and maximum margin matrix factorization provide similar results for factorization of binary matrices.

8.7 Generalized Low-Rank Models

The application of specialized forms of matrix factorization (like logistic and maximum margin factorization) for binary data leads to some interesting questions. What happens if the original data matrix contains entries of different types? In machine learning, it is common

Table 8.1: A demographic data set containing heterogeneous data types in different columns

Age (Numerical)	Gender (Binary)	Zip Code (Categorical)	Race (Categorical)	Education Level (Ordinal)
32	F	10598	Caucasian	Bachelors
41	M	10532	African American	Bachelors
36	M	10562	Filipino	High School
32	F	10532	Hispanic	Masters
29	F	10532	Native American	Doctorate

to encounter data matrices in which different features of the matrix might be numerical, binary, categorical, ordinal, and so on. Table 8.1 illustrates a table of demographic data containing *heterogeneous* data types in which the different columns correspond to different data types. A natural question that arises is how one might possibly create a factorization of a table with such bewilderingly different data types.

In this case, we have an $n \times d$ matrix $D = [x_{ij}]$ of heterogeneous data types and $W = [w_{ij}]$ is an $n \times d$ matrix of weights. An important distinction from the scenarios we have seen so far is that the data type of x_{ij} depends on the column index j . In order to perform the factorization, we use an $n \times k$ matrix U and an $r \times k$ matrix V . In most forms of factorization, the number of rows in V is equal to the number of columns d in D , whereas we have $r > d$ in this case. Why is $r > d$? The reason is that some data types like categorical data require multiple rows for a single column of D , which is not the case in any of the models we have seen so far. Therefore, the “reconstructed matrix” UV^T does not have the same size as the original matrix D ; the reconstruction has the same number of rows as D , but it might have a much larger number of columns. Therefore, a one-to-one correspondence of columns between D and UV^T is no longer possible, and it is sensitive to the data type at hand. The j th column in D is associated with multiple columns in UV^T , and it is assumed that these columns are located consecutively in UV^T with column indices in the range $[l_j, h_j]$. When the column index j of D corresponds to a numeric, ordinal, or binary variable, we will have $l_j = h_j$ and therefore a single column of UV^T corresponds to a single column of D . However, for some data types like categorical data, we will have $h_j > l_j$. For each column j in D , we define a loss function that is specific to the column at hand, and it uses $h_j - l_j + 2$ arguments. The loss function $\mathcal{L}_j(\cdot)$ of the j th column of D is defined as follows:

1. The first argument of the loss function for any entry (i, j) of the j th column of D is the observed value of the entry x_{ij} .
2. The remaining $h_j - l_j + 1$ arguments of the loss function use the $r = h_j - l_j + 1$ values $z_{i, l_j} \dots z_{i, l_j + r}$, where we have $z_{iq} = \bar{u}_i \cdot \bar{v}_q$ for each $q \in \{l_j \dots l_j + r - 1\}$.
3. The loss value L_{ij} for the (i, j) th entry of D is defined using the loss function $\mathcal{L}_j$ specific to column j :

$$L_{ij} = \mathcal{L}_j(x_{ij}, z_{i, l_j} \dots z_{i, l_j + r})$$

The nature of the loss function depends heavily on the data type at hand. We have already seen some examples of loss functions for binary and numerical variables. In the following, we will also introduce some loss functions for categorical and ordinal variables.

The overall objective function of the factorization can be expressed as a function of entry-specific weights and additional regularization:

$$\text{Minimize } J = \sum_{i=1}^n \sum_{j=1}^d w_{ij} L_{ij} + \frac{\lambda}{2} (\|U\|_F^2 + \|V\|_F^2)$$

We have already seen how the loss functions for numerical and binary factorization were derived directly from their counterparts in linear regression and binary classification. Correspondingly, we can also derive the loss functions of categorical and ordinal values from their counterparts in multinomial logistic regression and ordinal regression.

8.7.1 Handling Categorical Entries

The key point is that the modeling of categorical entries requires exactly as many entries in UV^T as the number of distinct values of the categorical attribute. Consider the j th column of D , which can take on $s_j = h_j - l_j + 1$ possible values $a_1 \dots a_{s_j}$. Then, the model of multinomial logistic regression computes the probability of the (i, j) th entry taking on the value a_r as follows:

$$P_{ij}(a_r) = \frac{\exp(z_{i,l_j+r-1})}{\sum_{s=1}^r \exp(z_{i,l_j+s-1})}$$

Then, the loss from this entry can be defined as follows:

$$L_{ij} = -\log [P_{ij}(x_{ij})]$$

This is the straightforward log-likelihood model of multinomial logistic regression discussed in Section 4.9.2 of Chapter 4.

8.7.2 Handling Ordinal Entries

Ordinal entries are those in which there are fixed number of ordered values that need to be predicted. However, the distances among these different entries are unknown. For example, Table 8.1 contains four possible education levels that are ordered. However, it is not easily possible to know the distances among the different education levels. Let T_j be the number of possible distinct values for the j th column of D (which happens to be an ordinal variable). We define $m = T_j - 1$ different ordered thresholds denoted by $y_1 \dots y_m$ (in increasing order). These thresholds, which are referred to as *intercepts*, also need to be learned in a data-driven manner, although they are not part of the matrices U and V . The matrix V contains a single row $\bar{v}_{o_j}$ with index o_j for this ordinal column. In traditional matrix factorization, the number of rows of V is equal to the number of columns in D because there is a one-to-one correspondence between the rows of V and columns of D . This is not the case for heterogeneous data tables containing categorical data types. Although ordinal data types require only one column in V , the value of o_j may be larger than j because some other types of data (like categorical data) in the same matrix require more than one row in V ; this can cause a persistent mismatch in the indices of the original matrix D and latent vector matrix V . The prediction of the (i, j) th entry of D is obtained by computing $\bar{u}_i \cdot \bar{v}_{o_j}$ and then predicting it as follows:

$$\hat{x}_{ij} = \begin{cases} \text{1st ordinal value} & z_{i,o_j} \leq y_1 \\ q\text{th ordinal value} & q \in [2, m], y_{q-1} \leq z_{i,o_j} \leq y_q \\ (m+1)\text{th ordinal value} & z_{i,o_j} > y_m \end{cases}$$

In other words, we use the ordered thresholds $y_1 \dots y_m$ to define $(m+1)$ buckets on the real line. The (i, j) th entry is mapped to an ordinal value depending on which bucket it falls in on the real line. In the following discussion, we also assume (for notational convenience) that $y_0 = -\infty$ and $y_{m+1} = +\infty$. Although these (trivial) end-point intercepts do not need to be learned, they help in reducing unnecessary case-wise analysis. For example, the prediction $\hat{x}_{ij}$ can now be collapsed into a single case as follows:

$$\hat{x}_{ij} = \left\{ q\text{th ordinal value} \quad q \in [1, m+1], y_{q-1} \leq z_{i,o_j} \leq y_q \right.$$

There are many possible ways in which one can set up the loss function for ordinal entries. One possible way is to use the *proportional odds model* in which we view the ordinal prediction model as that of summing the losses of m different binary predictions for the (i, j) th entry— the q th prediction checks whether x_{ij} and z_{i,o_j} end up on the same side of y_q . Note that this is the same approach used in binary logistic matrix factorization, except that we have to learn multiple intercepts $y_1 \dots y_m$ in this case. Then, we compute the probability that the (i, j) th entry lies on either side of y_b as follows:

$$P_{ij}(x_{ij} \leq y_b) = \frac{1}{1 + \exp(z_{i,o_j} - y_b)}$$

$$P_{ij}(x_{ij} > y_b) = \frac{1}{1 + \exp(-z_{i,o_j} + y_b)}$$

It is easy to verify that the sum of the above two probabilities is 1. Note that larger values of y_b will increase the probability $P_{ij}(x_{ij} \leq y_b)$, which makes sense in this case. At $b = 0$ and $b = m+1$, the values of y_b are fixed to $-\infty$ and $+\infty$, respectively. In such cases, it can be easily verified that each of the aforementioned probabilities is either a 0 or 1.

Suppose that the observed value of the ordinal variable x_{ij} lies between the current values of y_s and y_{s+1} for some $s \in \{0, \dots, m\}$. Then, we would like $P(x_{ij} > y_b)$ to be as large as possible for $b \leq s$ and we would like $P(x_{ij} \leq y_b)$ to be as large as possible for $b > s$. This is achieved with the use of the following loss function:

$$L_{ij} = - \sum_{b=1}^s \log[P_{ij}(x_{ij} > y_b)] - \sum_{b=s+1}^m \log[P_{ij}(x_{ij} \leq y_b)]$$

Note that if s is 0, the first set of terms vanish. Similarly, if s is m , the second set of terms vanish. This loss function is very similar to binary logistic prediction; the main difference from binary logistic modeling is that we have m different binary predictions corresponding to each threshold y_s , and we want to reward predictions on the correct side of each of m thresholds. The loss function contains the sum of m different (negative) rewards. Here, it is important to note that each y_s is a variable. Therefore, the gradient-descent procedure not only has to update the factor matrices, but it also has to update the thresholds $y_1 \dots y_m$.

In all problems we have seen so far, one can always substitute hinge loss wherever logistic loss is used. This is because of the similarity of these loss functions (cf. Figure 4.9 of Chapter 4). Suppose that the ordinal variable x_{ij} lies between the current values of y_s and y_{s+1} for some $s \in \{0, \dots, m\}$. As in the case of logistic model, we can view the loss function as the sum of m different losses for the m different binary predictions (one for each non-trivial threshold y_b). The loss function penalizes cases in which z_{i,o_j} either lies on the

wrong side of each y_b , or it lies on the correct side (but without sufficient margin). This is achieved by defining the loss function L_{ij} as follows:

$$L_{ij} = \sum_{b=1}^s \max(1 - z_{i,o_j} + y_b, 0) + \sum_{b=s+1}^m \max(1 + z_{i,o_j} - y_b, 0)$$

The hinge loss has the advantage of having a simpler derivative. This model is also available in the Julia package discussed in [128].

8.8 Shared Matrix Factorization

Shared matrix factorization is used in order to factorize multiple matrices simultaneously. Furthermore, the matrices have at least one modality in common. For example, if we have a graph of objects in which each node (object) of the graph also contains a document, then we have two matrices corresponding to the graph connectivity structure and the documents associated with all the nodes. Furthermore, since there is one-to-one correspondence between each node and a document, it is possible to create two matrices in which one of the dimensions is common between the two matrices. This sharing of a data modality is critical in being able to perform shared matrix factorization.

The basic idea in shared matrix factorization is to perform the factorization of two matrices with a *shared dimension* by using a *shared factor matrix*. Consider two matrices D and M that are of sizes $n \times d$ and $n \times m$, respectively. Because of the shared modality, the numbers of rows in the two matrices are the same; each row of D corresponds to a row in M . For example, D might correspond to a document-term matrix (which contains frequencies of words in documents) and M might correspond to a document-user matrix (which contains binary information on users placing a “like” rating on one or more documents). Therefore, the values are either 0 or 1 depending on whether or not a user has placed a like rating on an item. This is a classical example of implicit feedback data. An important property of the two matrices is that the i th row of D and the i th row of M correspond to the same object (which is a document in this case). Note that this one-to-one correspondence is essential for shared matrix factorization to work. Ideally, one would like to create a latent representation of documents based on *both* the content-based and the feedback data. This type of feature-engineered representation is helpful in a variety of tasks such as recommendations. For example, a similarity search using such an engineered representation will provide an output that takes both the topical and the interest behavior into account.

We introduce the shared $n \times k$ factor matrix U for documents, the $d \times k$ factor matrix V for words, and the $m \times k$ factor matrix W for users. Then, we would like to perform the following set of shared factorizations:

$$D \approx UV^T \quad \text{Document-Word Factorization}$$

$$M \approx UW^T \quad \text{Document-User Factorization}$$

One can then set up the objective function of the factorization as follows:

$$\text{Minimize } J = \frac{1}{2} \|D - UV^T\|_F^2 + \frac{\beta}{2} \|M - UW^T\|_F^2 + \underbrace{\frac{\lambda}{2} (\|U\|_F^2 + \|V\|_F^2 + \|W\|_F^2)}_{\text{Regularization}}$$

Here, $\beta > 0$ provides the relative weights of the two factorizations, and $\lambda > 0$ is the regularization parameter. The value of β is often chosen based on application-specific considerations.

8.8.1 Gradient Descent Steps for Shared Factorization

In the following, we will derive the gradient-descent steps for the matrix factorization model discussed in the previous section. We compute the gradient of J with respect to the entries in U , V , and W . For any current values of U , V , and W , let e_{ij}^D represent the (i, j) th entry of the error matrix $(D - UV^T)$, and e_{ij}^M represent the (i, j) th entry of the error matrix $(M - UW^T)$. The partial derivatives of J are as follows:

$$\begin{aligned}\frac{\partial J}{\partial u_{iq}} &= -\sum_{j=1}^d e_{ij}^D v_{jq} - \beta \sum_{p=1}^m e_{ip}^M w_{pq} + \lambda u_{iq} \quad \forall i \in \{1 \dots n\}, \forall q \in \{1 \dots k\} \\ \frac{\partial J}{\partial v_{jq}} &= -\sum_{i=1}^n e_{ij}^D u_{iq} + \lambda v_{jq} \quad \forall j \in \{1 \dots d\}, \forall q \in \{1 \dots k\} \\ \frac{\partial J}{\partial w_{pq}} &= -\beta \sum_{i=1}^n e_{ip}^M u_{iq} + \lambda w_{pq} \quad \forall p \in \{1 \dots m\}, \forall q \in \{1 \dots k\}\end{aligned}$$

These gradients can be used to update the entire set of $(n + m + d)k$ parameters with a step-size of α . This approach corresponds to vanilla gradient descent. It is also possible to use *stochastic* gradient descent, which effectively computes the gradients with respect to residual errors in randomly sampled *entries* of the matrices. One can sample any entry in *either* the document-term matrix or the adjacency matrix, and then perform the gradient-descent step with respect to the error in this single entry:

Randomly sample any entry from either D or M ;
Perform a gradient-descent step with respect to entry-specific loss;

The probability of sampling each entry is fixed irrespective of which matrix it is drawn from. Consider a case in which the (i, j) th entry in the document-term matrix is sampled with error e_{ij}^D . Then, the following updates are executed for each $q \in \{1 \dots k\}$ and step-size α :

$$\begin{aligned}u_{iq} &\leftarrow u_{iq}(1 - \alpha \cdot \lambda/2) + \alpha e_{ij}^D v_{jq} \quad \forall q \in \{1 \dots k\} \\ v_{jq} &\leftarrow v_{jq}(1 - \alpha \cdot \lambda) + \alpha e_{ij}^D u_{iq} \quad \forall q \in \{1 \dots k\}\end{aligned}$$

On the other hand, if the (i, p) th entry in the adjacency matrix is sampled, then the following updates are performed for each $q \in \{1 \dots k\}$ and step-size α :

$$\begin{aligned}u_{iq} &\leftarrow u_{iq}(1 - \alpha \cdot \lambda/2) + \alpha \beta e_{ip}^M w_{pq} \quad \forall q \in \{1 \dots k\} \\ w_{pq} &\leftarrow w_{pq}(1 - \alpha \cdot \lambda) + \alpha \beta e_{ip}^M u_{iq} \quad \forall q \in \{1 \dots k\}\end{aligned}$$

These steps are repeated to convergence.

8.8.2 How to Set Up Shared Models in Arbitrary Scenarios

Shared matrix factorization is useful in any scenario where the matrices are drawn from heterogeneous domains, but they share at least some connections with one another. The connections can themselves be expressed as matrices in some cases. For example, one might have completely independent matrices for feature representations of documents and images, and a third matrix might contain hyperlinks from documents to images. In such a case, one

would have an $n_1 \times d_1$ matrix D_1 for document-term representation, an $n_2 \times d_2$ matrix D_2 for image-(visual word) representation, and an $n_1 \times n_2$ matrix A containing connections between documents and visual words. Corresponding, we need two factor matrices U_1 and U_2 for documents and images, and two factor matrices V_1 and V_2 for text-words and visual-words. Therefore, one would like to perform the factorization as follows:

$$\begin{aligned} D_1 &\approx U_1 V_1^T && \text{Document-Word Factorization} \\ D_2 &\approx U_2 V_2^T && \text{Image-(Visual Word) Factorization} \\ A &\approx U_1 U_2^T && \text{Connection-Matrix Factorization} \end{aligned}$$

One can set up an objective function that minimizes the sum of the squares of the errors over all three matrices. It is even possible to weight the different types of errors differently, depending on the application at hand. We leave the derivation of the gradient descent steps as an exercise for the reader.

Problem 8.8.1 *Write down the sum-of-squared objective function for the factorization of the matrices D_1 , D_2 , and A , as discussed above. Derive the gradient descent steps for the entries in these matrices. You may introduce any notation as needed for this problem.*

All the settings used for shared matrix factorization are very similar; *we have a set of matrices in which some of the modalities are shared*, and we wish to extract latent representations of the shared relationships implicit in these matrices. The key in this entire process is to use *shared latent factors* between different modalities so that they are able to incorporate the impact of these relationships in an indirect (i.e., latent) way within the extracted embedding. A single set of factors is introduced for each shared modality, and each matrix is factorized. The sum-of-squared objective function is used to determine the gradient-descent updates.

8.9 Factorization Machines

Factorization machines are closely related to shared matrix factorization methods, and are particularly suitable when each data instance contains features from multiple domains. For example, consider an item that is tagged with particular keywords by a user and also rated by that user. In such a case, the feature set corresponds to all the item identifiers, all the possible keywords, and the user identifiers. The feature values of the user identifier, item identifier, and the relevant keywords are set to 1, whereas all other feature values are set to 0. The dependent variable is equal to the value of the rating.

Factorization machines are polynomial regression techniques, in which strong regularization conditions are imposed on the regression coefficients in order to handle the challenges of sparsity. Sparsity is common in short-text domains, such as the social content on bulletin boards, social network datasets, and chat messengers. It is also common in recommender systems.

An example of a data set drawn from the recommendation domain is illustrated in Figure 8.6. It is evident that there are three types of attributes corresponding to user attributes, item attributes, and tagging keywords. Furthermore, the rating corresponds to the dependent variable, which is also the regressand. At first sight, this data set seems to be no different from a traditional multidimensional data set to which one might apply least-squares regression in order to model the rating as a linear function of the regressors.

$$\hat{y}(\bar{x}) = b + \sum_{i=1}^p w_i x_i + \sum_{i=1}^p \sum_{j=i+1}^p s_{ij} x_i x_j \quad (8.19)$$

Note that one could also include the second-order term $\sum_{i=1}^p s_{ii} x_i^2$, although x_i is often drawn from sparse domains with little variation in non-zero values of x_i , and the addition of such a term is not always helpful. For example, if the value of x_i is binary (as is common), the coefficient of x_i^2 would be redundant with respect to that of x_i .

One observation is that the above model is very similar to what one would obtain with the use of kernel regression with a second-order polynomial kernel. In sparse domains like text, such kernels often overfit the data, especially when the dimensionality is large and the data is sparse. Even for an application in a single domain (e.g., short-text tweets), the value of d is greater than 10^5 , and therefore the number of second-order coefficients is more than 10^{10} . With any training data set containing less than 10^{10} points, one would perform quite poorly. This problem is exacerbated by sparsity, in which pairs of attributes co-occur rarely in the training data, and may not generalize to the test data. For example, in a recommender application, a particular user-item pair may occur only once in the entire training data, and it will not occur in the test data if it occurs in the training data. In fact, all the user-item pairs that occur in the test data will not have occurred in the training data. How, then, does one learn the interaction coefficients s_{ij} for such user-item pairs? Similarly, in a short-text mining application, the words “movie” and “film” may occur together, and the words “comedy” and “film” may also occur together, but the words “comedy” and “movie” might never have occurred together in the training data. What does one do, if the last pair occurs in the test data?

A key observation is that one can use the learned values of s_{ij} for the other two pairs (i.e., “comedy”/“film” and “movie”/“film”) in order to make some inferences about the interaction coefficient for the pair “comedy” and “movie.” How does one achieve this goal? The key idea is to assume that the $d \times d$ matrix $S = [s_{ij}]$ of second-order coefficients has a *low-rank* structure for some $d \times k$ matrix $V = [v_{is}]$:

$$S = VV^T \quad (8.20)$$

Here, k is the rank of the factorization. Intuitively, one can view Equation 8.20 as a kind of regularization constraint on the (massive number of) second-order coefficients in order to prevent overfitting. Therefore, if $\bar{v}_i = [v_{i1} \dots v_{ik}]$ is the k -dimensional row vector representing the i th row of V , we have:

$$s_{ij} = \bar{v}_i \cdot \bar{v}_j \quad (8.21)$$

By substituting Equation 8.21 in the prediction function of Equation 8.19, one obtains the following:

$$\hat{y}(\bar{x}) = b + \sum_{i=1}^p w_i x_i + \sum_{i=1}^p \sum_{j=i+1}^p (\bar{v}_i \cdot \bar{v}_j) x_i x_j \quad (8.22)$$

The variables to be learned are b , the different values of w_i , and each of the vectors $\bar{v}_i$. Although the number of interaction terms might seem large, most of them will evaluate to zero in sparse settings in Equation 8.22. This is one of the reasons that factorization machines are designed to be used only in sparse settings where most of the terms of Equation 8.22 evaluate to 0. A crucial point is that we only need to learn the $O(d \cdot k)$ parameters represented by $\bar{v}_1 \dots \bar{v}_k$ in lieu of the $O(d^2)$ parameters in $[s_{ij}]_{d \times d}$.

A natural approach to solve this problem is to use the stochastic gradient-descent method, in which one cycles through the observed values of the dependent variable to

compute the gradients with respect to the error in the observed entry. The update step with respect to any particular model parameter $\theta \in \{b, w_i, v_{is}\}$ depends on the error $e(\bar{x}) = y(\bar{x}) - \hat{y}(\bar{x})$ between the predicted and observed values:

$$\theta \leftarrow \theta(1 - \alpha \cdot \lambda) + \alpha \cdot e(\bar{x}) \frac{\partial \hat{y}(\bar{x})}{\partial \theta} \quad (8.23)$$

Here, $\alpha > 0$ is the learning rate, and $\lambda > 0$ is the regularization parameter. The partial derivative in the update equation is defined as follows:

$$\frac{\partial \hat{y}(\bar{x})}{\partial \theta} = \begin{cases} 1 & \text{if } \theta \text{ is } b \\ x_i & \text{if } \theta \text{ is } w_i \\ x_i \sum_{j=1}^p v_{js} \cdot x_j - v_{is} \cdot x_i^2 & \text{if } \theta \text{ is } v_{is} \end{cases} \quad (8.24)$$

The term $L_s = \sum_{j=1}^p v_{js} \cdot x_j$ in the third case is noteworthy. To avoid redundant effort, this term can be pre-stored while evaluating $\hat{y}(\bar{x})$ for computation of the error term $e(\bar{x}) = y(\bar{x}) - \hat{y}(\bar{x})$. This is because Equation 8.22 can be algebraically rearranged as follows:

$$\begin{aligned} \hat{y}(\bar{x}) &= b + \sum_{i=1}^p w_i x_i + \frac{1}{2} \sum_{s=1}^k \left(\left[\sum_{j=1}^p v_{js} \cdot x_j \right]^2 - \sum_{j=1}^p v_{js}^2 \cdot x_j^2 \right) \\ &= b + \sum_{i=1}^p w_i x_i + \frac{1}{2} \sum_{s=1}^k \left(L_s^2 - \sum_{j=1}^p v_{js}^2 \cdot x_j^2 \right) \end{aligned}$$

Furthermore, the parameters $\bar{v}_i$ and w_i do not need to be updated when $x_i = 0$. This allows for an efficient update process in sparse settings, which is linear in both the number of non-zero entries and the value of k .

Factorization machines can be used for any (massively sparse) classification or regression task; ratings prediction in recommender systems is only one example of a natural application. Although the model is inherently designed for regression, binary classification can be handled by applying the logistic function on the numerical predictions to derive the probability whether $\hat{y}(\bar{x})$ is +1 or -1. The prediction function of Equation 8.22 is modified to a form used in logistic regression:

$$P[y(\bar{x}) = 1] = \frac{1}{1 + \exp(-[b + \sum_{i=1}^p w_i x_i + \sum_{i=1}^p \sum_{j=i+1}^p (\bar{v}_i \cdot \bar{v}_j) x_i x_j])} \quad (8.25)$$

This form is the same as the logistic regression approach discussed in Chapter 4. The main difference is that we are also using second-order interactions within the prediction function. A log-likelihood criterion can be optimized to learn the underlying model parameters with a gradient-descent approach [47, 107, 108].

The description in this section is based on second-order factorization machines that are popularly used in practice. In third-order polynomial regression, we would have $O(p^3)$ additional regression coefficients of the form w_{ijk} , which correspond to interaction terms of the form $x_i x_j x_k$. These coefficients would define a massive third-order *tensor*, which can be compressed with tensor factorization. Although higher-order factorization machines have also been developed, they are often impractical because of greater computational complexity and overfitting. A software library, referred to as *libFM* [108], provides an excellent set of factorization machine implementations. The main task in using *libFM* is an initial feature

engineering effort, and the effectiveness of the model mainly depends on the skill of the analyst in extracting the correct set of features. Other useful libraries include *fastFM* [11] and³ *libMF* [144], which have some fast learning methods for factorization machines.

8.10 Summary

Matrix factorization is one of the most fundamental tools in machine learning, which is exploited both for the useful linear algebra and also for the compression properties of the underlying factors. One of the most fundamental forms of factorization is singular value decomposition in which the columns of the different factor matrices are mutually orthogonal. More general forms of the matrix factorization modify the optimization model to allow different types of objective functions, constraints, and data types. Certain types of constraints like nonnegativity have a regularization effect, and they help in creating more interpretable matrix factorizations. Methods like logistic matrix factorization, maximum margin factorization, and generalized low-rank models are designed to deal with different data types. Shared matrix factorization and factorization machines are designed to factorize multiple matrices. In general, the broader theme of matrix factorization provides a very wide variety of tools that can be harnessed for various machine learning scenarios.

8.11 Further Reading

Discussions of SVD and unconstrained matrix factorization may be found in many books on linear algebra [77, 122, 123, 130]. The use of unconstrained matrix factorization for recommender systems is discussed in detail in [3, 75]. The use of coordinate descent for matrix factorization in recommender systems is discussed in [137]. Alternating least-squares methods are discussed in [69, 141].

Nonnegative matrix factorization is introduced in [79], and its probabilistic counterpart, PLSA, is discussed in [63]. The relationship between the I-divergence objective function and PLSA is discussed in [35, 50]. The importance of deriving separate weight and value matrices for factorization from implicit feedback data is discussed in [65]. The application of various types of sparse factorization models to text and graph feature engineering is discussed in [2, 55, 91, 92, 101, 103]. The use of logistic matrix factorization for implicit feedback data is discussed in [70], and that of maximum margin matrix factorization is discussed in [120]. However, the presentation of maximum margin matrix factorization is much more simplified in this chapter. Generalized low-rank models are introduced in [128]. Regression models for ordinal data are introduced in [90]. An overview of different types of shared matrix factorization models is provided in [2, 3, 117]. Factorization machines are discussed in [107, 108].

8.12 Exercises

1. **Biased matrix factorization:** Consider the factorization of an incomplete $n \times d$ matrix D into an $n \times k$ matrix U and a $d \times k$ matrix V :

$$D \approx UV^T$$

³The libraries *libFM* and *libMF* are different.

Suppose you add the constraint that all entries of the penultimate column of U and the final column of V are fixed to 1. Discuss the similarity of this model to that of the addition of bias to classification models. How is gradient descent modified?

2. In the scenario of Exercise 1, will the Frobenius norm on observed ratings be better optimized with or without constraints on the final columns of U and V ? Why might it be desirable to add such a constraint during the estimation of missing entries?
3. Suppose that you have a symmetric $n \times n$ matrix D of similarities, which has missing entries. You decide to recover the missing entries by using the *symmetric factorization* $D \approx UU^T$. Here, U is an $n \times k$ matrix, and k is the rank of the factorization.
 - (a) Write the objective function for the optimization model using the Frobenius norm and L_2 -regularization.
 - (b) Derive the gradient-descent steps in terms of matrix-centric updates.
 - (c) Discuss the conditions under which an exact factorization will not exist, irrespective of how large a value of k is used for the factorization.
4. Derive the gradient-descent updates for L_1 -loss matrix factorization in which the objective function is $J = \|D - UV^T\|_1$.
5. Derive the gradient-descent updates for L_2 -loss matrix factorization in which L_1 -regularization is used on the factors.
6. In SVD, it is easy to compute the representation of out-of-sample matrices because of the orthonormality of the basis $d \times k$ matrix V . If the SVD factorization of the $n \times d$ matrix D is $D \approx UV^T$, then one can compute the representation of an out-of-sample $m \times d$ matrix D_o as $D_o V$. Show how you can efficiently compute a similar out-of-sample representation of D_o , when you are given a non-orthonormal factorization $D = UV^T$. Assume that m and k are much smaller than n and d .
7. Show that the k -means formulation in Section 4.10.3 of Chapter 4 is identical to the formulation of Section 8.2.1. [Hint: Propose a one-to-one mapping of optimization variables in the two problems. Show that the constraints and the objective functions are equivalent in the two cases.]
8. **Orthogonal Nonnegative Matrix Factorization:** Consider a nonnegative $n \times d$ data matrix D in which we try to approximately factorize D as UV^T with the Frobenius norm as the objective function. Suppose you add nonnegativity constraints on U and V along with the constraint $U^T U = I$. How many entries in each row of U will be non-zero? Discuss how you can extract a clustering from this factorization. Show that this approach is closely related to the k -means optimization formulation.
9. Suppose that you use GloVe on a quantity matrix $Q = [q_{ij}]$ in which each count q_{ij} is either 0 or 10000. A sizeable number of counts are 0s. Show that GloVe can discover a trivial factorization with zero error in which each word has the same embedded representation.
10. Derive the gradient update equations for using factorization machines in binary classification with logistic loss and hinge loss.

11. Suppose you want to perform the rank- k factorization $D \approx UV^T$ of the $n \times d$ matrix D using gradient descent. Propose an initialization method for U and V using QR decomposition of k randomly chosen columns of D .
12. Suppose that you have a sparse non-negative matrix D of size $n \times d$. What can you say about the dot product of any pair of columns as a consequence of sparsity? Use this fact along with the intuition derived from the previous exercise to initialize U using k randomly sampled columns of D for *non-negative matrix factorization*. In this case, the initialized matrices U and V need to be non-negative.
13. **Nonlinear matrix factorization of positive matrices:** Consider a nonlinear model for matrix factorization of positive matrices $D = [x_{ij}]$, where $D = F(UV^T)$, and $F(x) = x^2$ is applied in element-wise fashion. The vectors $\bar{u}_i$ and $\bar{v}_j$ represent the i th and j th rows of U and V , respectively. The loss function is $\|D - F(UV^T)\|_F^2$. Show that the gradient descent steps are as follows:

$$\begin{aligned}\bar{u}_i &\leftarrow \bar{u}_i + \alpha \sum_j (\bar{u}_i \cdot \bar{v}_j)(x_{ij} - F(\bar{u}_i \cdot \bar{v}_j))\bar{v}_j \\ \bar{v}_j &\leftarrow \bar{v}_j + \alpha \sum_i (\bar{u}_i \cdot \bar{v}_j)(x_{ij} - F(\bar{u}_i \cdot \bar{v}_j))\bar{u}_i\end{aligned}$$

14. **Out-of-sample factor learning:** Suppose that you learn the optimal matrix factorization $D \approx UV^T$ of $n \times d$ matrix D , where U, V are $n \times k$ and $d \times k$ matrices, respectively. Now you are given a new out-of-sample $t \times d$ data matrix D_o with rows collected using the same methodology as the rows of D (and with the same d attributes). You are asked to quickly factorize this out-of-sample data matrix into $D_o \approx U_o V^T$ with the objective of minimizing $\|D_o - U_o V^T\|_F^2$, where V is *fixed* to the matrix learned from the earlier in-sample factorization. Show that the problem can be decomposed into t linear regression problems, and the optimal solution U_o is given by:

$$U_o^T = V^+ D_o^T$$

Here, V^+ is the pseudoinverse of V . Show that the rank- k approximation of $D_o \approx U_o V^T$ is given by $D_o P_v$, where $P_v = V(V^T V)^{-1} V^T$ is the $d \times d$ projection matrix induced by V . Propose a fast solution approach using QR decomposition of V and back-substitution with a triangular equation system. How does this problem relate to the alternating minimization approach?

15. **Out-of-sample factor learning:** Consider the same scenario as Exercise 14, where you are trying to learn the out-of-sample factor matrix U_o for in-sample data matrix $D \approx UV^T$ and out-of-sample data matrix D_o . The factor matrix V is fixed from in-sample learning. Closed-form solutions, such as the one in Exercise 14, are rare in most matrix factorization settings. Discuss how the gradient-descent updates discussed in this chapter can be modified so that U_o can be learned directly. Specifically discuss the case of (i) unconstrained matrix factorization, (ii) nonnegative matrix factorization, and (iii) logistic matrix factorization.
16. Suppose that you have a user-item ratings matrix with numerical/missing values. Furthermore, users have rated each other's trustworthiness with binary/missing values.
- Show how you can use shared matrix factorization for estimating the rating of a user on an item that they have not already rated.
 - Show how you can use factorization machines to achieve similar goals as (a).

17. Propose an algorithm for finding outlier entries in a matrix with the use of matrix factorization.
18. Suppose that you are given the linkage of a large Website with n pages, in which each page contains a bag of words drawn from a lexicon of size d . Furthermore, you are given information on how m users have rated each page on a scale of 1 to 5. The ratings data is incomplete. Propose a model to create an embedding for each Webpage by combining all three pieces of information. [Hint: This is a shared matrix factorization problem.]
19. **True or false:** A zero error non-negative matrix factorization (NMF) UV^T of an $n \times d$ non-negative matrix D always exists, where U is an $n \times k$ matrix and V is a $d \times k$ matrix, as long as k is chosen large enough. At what value of k can you get an exact NMF of the following matrix?

$$D = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

20. **True or false:** Suppose you have the exact non-negative factorization (NMF) UV^T of a matrix D , so that each column of V is constrained to sum to 1. Subject to this normalization rule, the NMF of D is unique.
21. Discuss why the following algorithm will work in computing the matrix factorization $D_{n \times d} \approx UV^T$ after initializing $U_{n \times k}$ and $V_{d \times k}$ randomly:

repeat; $U \leftarrow DV^+$; $V \leftarrow D^T U^+$; **until convergence;**
22. Derive the gradient-descent updates of unconstrained matrix factorization with L_1 -regularization. You may assume that the regularization parameter is $\lambda > 0$.
23. **Alternating nonnegative least-squares:** Propose an algorithm for nonnegative matrix factorization using the alternating least-squares method [Hint: See nonnegative regression in Chapter 6.]
24. **Bounded matrix factorization:** In bounded matrix factorization, the entries of U and V in the factorization $D \approx UV^T$ are bounded above and below by specific values. Propose a computational algorithm for bounded matrix factorization using (i) gradient descent, and (ii) alternating least-squares.
25. Suppose that you have a very large and dense matrix D of low rank that you cannot hold in memory, and you want to factorize it as $D \approx UV^T$. Propose a method for factorization that uses only sparse matrix multiplication. [Hint: Read the section on recommender systems.]
26. **Temporal matrix factorization:** Consider a sequence of $n \times d$ matrices $D_1 \dots D_t$ that are slowly evolving over t time stamps. Show how one can create an optimization model to infer a single $n \times k$ *static* factor matrix that does not change over time, and multiple $d \times k$ *dynamic* factor matrices, each of which is time-specific. Derive the gradient descent steps to find the factor matrices.